

CS5489 - Machine Learning

Lecture 10a - Auto-Encoders and Deep Generative Models

Dept. of Computer Science, City University of Hong Kong

Outline

- Unsupervised models
 - Autoencoders
 - Denoising Autoencoders
- Deep generative models
 - Variational Autoencoders
 - GANs
 - Diffusion models

```
In [1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib
import numpy as np
from sklearn import *
from scipy import stats
from scipy import signal
import random
import math
from prettytable import PrettyTable

rbow = plt.get_cmap('rainbow')
```

```
In [2]: import torch
import torchvision
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torchvision import transforms
from torchvision.transforms import ToTensor
```

```

from torchvision.transforms import functional as F
import logging
logging.basicConfig()
import struct
import sys
print(f"Python: {sys.version} PyTorch: {torch.__version__}")

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

```

Python: 3.13.7 | packaged by Anaconda, Inc. | (main, Sep 9 2025, 19:54:17) [Clang 17.0.6] PyTorch: 2.8.0
Using device: cpu

```

In [3]: def plot_history(history):
        fig, ax1 = plt.subplots()

        ax1.plot(history['train_loss'], 'r', label="training loss ({:.6f})")
        ax1.plot(history['val_loss'], 'r--', label="validation loss ({:.6f})")
        ax1.grid(True)
        ax1.set_xlabel('iteration')
        ax1.legend(loc="best", fontsize=9)
        ax1.set_ylabel('loss', color='r')
        ax1.tick_params('y', colors='r')

        if 'train_accuracy' in history:
            ax2 = ax1.twinx()

            ax2.plot(history['train_accuracy'], 'b', label="training acc ({:.6f})")
            ax2.plot(history['val_accuracy'], 'b--', label="validation acc ({:.6f})")

            ax2.legend(loc="best", fontsize=9)
            ax2.set_ylabel('acc', color='b')
            ax2.tick_params('y', colors='b')
        plt.show()

```

```

In [4]: def show_imgs(W_list, nc=10, highlight_green=None, highlight_red=None,
                    nfilter = len(W_list)
                    nr = (nfilter - 1) // nc + 1
                    for i in range(nr):
                        for j in range(nc):
                            idx = i * nc + j
                            if idx == nfilter:
                                break
                            plt.subplot(nr, nc, idx + 1)
                            cur_W = W_list[idx]
                            if np.ndim(cur_W) == 2:
                                plt.imshow(cur_W, cmap='gray', interpolation='nearest')
                            else:
                                plt.imshow(cur_W, interpolation='nearest')

                            if titles is not None:

```

```

        if isinstance(titles, str):
            plt.title(titles % idx)
        elif isinstance(titles, list):
            plt.title(titles[idx])
        else:
            plt.title(titles)

    if ((highlight_green is not None) and highlight_green[idx]
        ((highlight_red is not None) and highlight_red[idx])):
        ax = plt.gca()
        if highlight_green[idx]:
            mycol = '#00FF00'
        else:
            mycol = 'r'
        for S in ['bottom', 'top', 'right', 'left']:
            ax.spines[S].set_color(mycol)
            ax.spines[S].set_lw(2.0)
        ax.xaxis.set_ticks_position('none')
        ax.yaxis.set_ticks_position('none')
        ax.set_xticks([])
        ax.set_yticks([])
    else:
        plt.gca().set_axis_off()

plt.show()

```

In [5]:

```

def read_32int(f):
    return struct.unpack('>I', f.read(4))[0]
def read_img(img_path):
    with open(img_path, 'rb') as f:
        magic_num = read_32int(f)
        num_image = read_32int(f)
        n_row = read_32int(f)
        n_col = read_32int(f)
        #print 'num_image = {}; n_row = {}; n_col = {}'.format(num_image,
        res = []
        npixel = n_row * n_col
        res_arr = np.fromfile(f, dtype='B')
        res_arr = res_arr.reshape((num_image, n_row, n_col), order='C')
        #print 'image data shape = {}'.format(res_arr.shape)
        return num_image, n_row, n_col, res_arr
def read_label(label_path):
    with open(label_path, 'rb') as f:
        magic_num = read_32int(f)
        num_label = read_32int(f)
        #print 'num_label = {}'.format(num_label)
        res_arr = np.fromfile(f, dtype='B')
        #print res_arr.shape
        #res_arr = res_arr.reshape((num_label, 1))
        res_arr = res_arr.ravel()
        #print 'label data shape = {}'.format(res_arr.shape)
        return num_label, res_arr

```

```
In [6]: n_train, nrow, ncol, training = read_img('data/train-images.idx3-ubyte',
_, trainY = read_label('data/train-labels.idx1-ubyte')
n_test, _, _, testing = read_img('data/t10k-images.idx3-ubyte')
_, testY = read_label('data/t10k-labels.idx1-ubyte')

# for demonstration we only use 10% of the training data
sample_index = range(0, training.shape[0], 10)
training = training[sample_index]
trainY = trainY[sample_index]
print(training.shape)
print(trainY.shape)
print(testing.shape)
print(testY.shape)
```

```
(6000, 28, 28)
(6000,)
(10000, 28, 28)
(10000,)
```

```
In [7]: # Convert to PyTorch tensors and scale to 0-1
trainI = torch.from_numpy(training.reshape((6000, 1, 28, 28)) / 255.0)
testI = torch.from_numpy(testing.reshape((10000, 1, 28, 28)) / 255.0).

print(trainI.shape)
print(testI.shape)
```

```
torch.Size([6000, 1, 28, 28])
torch.Size([10000, 1, 28, 28])
```

```
In [8]: # Generate fixed validation set
from sklearn.model_selection import train_test_split

vtrainI_np, validI_np, vtrainY_np, validY_np = train_test_split(
    trainI.numpy(), trainY,
    train_size=0.9, test_size=0.1, random_state=4487
)

vtrainI = torch.from_numpy(vtrainI_np).float()
validI = torch.from_numpy(validI_np).float()
vtrainY = torch.from_numpy(vtrainY_np).long()
validY = torch.from_numpy(validY_np).long()

validsetI = (validI, validY)

print(f"Training set: {vtrainI.shape}, {vtrainY.shape}")
print(f"Validation set: {validI.shape}, {validY.shape}")

cid, cfreq = vtrainY.unique(return_counts=True)

print(f"Training set Distribution:", cfreq.numpy().tolist())
```

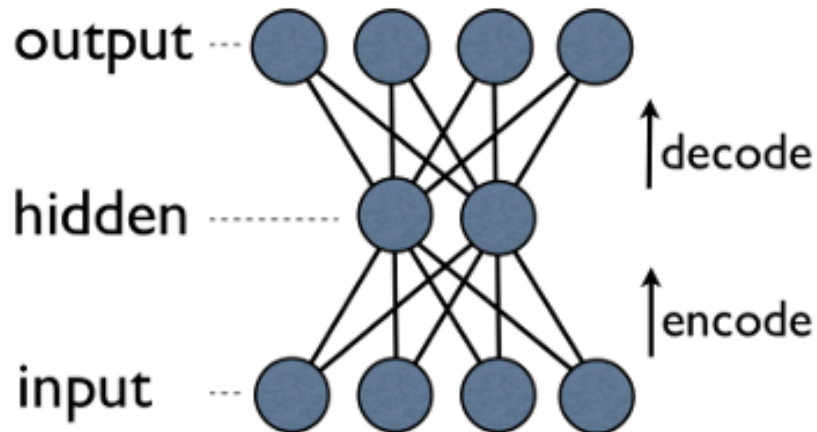
Training set: `torch.Size([5400, 1, 28, 28])`, `torch.Size([5400])`
Validation set: `torch.Size([600, 1, 28, 28])`, `torch.Size([600])`
Training set Distribution: [491, 599, 554, 556, 513, 518, 567, 584, 496, 522]

Neural Networks and Unsupervised Learning

- How to use NN for dimensionality reduction or clustering?

Autoencoder

- learn a latent representation of the data.
- Use the hidden layer as the lower-dimensional representation ("code", "bottleneck" layer)
- Train the network to "encode" and "decode"
 - run it through the encoding-decoding network
 - minimize the difference between the output and the original input



Autoencoder

- encode and decode with NN:
 - encoder: $\mathbf{z} = f(\mathbf{x})$
 - decoder: $\hat{\mathbf{x}} = g(\mathbf{z})$
- reconstruction: $\hat{\mathbf{x}} = g(f(\mathbf{x}))$
- Goal: minimize the reconstruction loss.
 - MSE (real inputs): $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$
 - binary cross-entropy (inputs bounded in $[0,1]$):

$$\mathbf{x} \log \hat{\mathbf{x}} + (1 - \mathbf{x}) \log(1 - \hat{\mathbf{x}})$$

Weight Sharing

- to reduce the number of parameters, share weights between the two layers
- D's weights are the transpose of E's weights
 - encoder: $\mathbf{z} = f(\mathbf{x}) = h_1(\mathbf{A}^T \mathbf{x})$
 - decoder: $\hat{\mathbf{x}} = g(\mathbf{z}) = h_2(\mathbf{A} \mathbf{z})$
- if h_1 and h_2 are linear activations, then it's similar to PCA.
 - except the orthogonal constraint on $\mathbf{A}$
- use more layers/non-linearities gives more general latent representations.

Example on MNIST

- Define the Dense layer using transposed weights from another layer.
 - has its own bias term
 - Define encoder, decoder, and the combination

```
In [9]: # Define the Autoencoder
class DenseAE(nn.Module):
    def __init__(self):
        super(DenseAE, self).__init__()
        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(784, 10),
            nn.ReLU()
        )

        # Decoder using DenseTranspose
        self.decoder = nn.Sequential(
            nn.Linear(10, 784),
            nn.Sigmoid()
        )

        self.initialize()

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

    def initialize(self):
        nn.init.zeros_(self.encoder[0].bias)
        nn.init.zeros_(self.decoder[0].bias)
```

```

In [10]: # Model Parameter Summarization
def summarize_layer_parameters(model):
    """
    Summarizes the number of parameters in each layer of a PyTorch model
    """
    trainable_table = PrettyTable(["Module Name", "Parameters"])
    untrainable_table = PrettyTable(["Module Name", "Parameters"])
    total_params = {'train':0, 'untrain':0}
    for name, parameter in model.named_parameters():
        if parameter.requires_grad: # Only count trainable parameters
            params = parameter.numel()
            trainable_table.add_row([name, params])
            total_params['train'] += params
        else: # Only count untrainable parameters
            params = parameter.numel()
            untrainable_table.add_row([name, params])
            total_params['untrain'] += params

    print(trainable_table)
    print(f"\nTotal Trainable Parameters: {total_params['train']}")
    if total_params['untrain'] > 0:
        print(untrainable_table)
    print(f"\nTotal Un-Trainable Parameters: {total_params['untrain']}")

```

```

In [11]: # Early stopping implementation
class EarlyStopping:
    def __init__(self, patience=10, min_delta=0.0001, verbose=1):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.current_loss = 1.0
        self.early_stop = False
        self.verbose = verbose

    def __call__(self, current_loss):
        if self.current_loss - current_loss > self.min_delta:
            self.current_loss = current_loss
            self.counter = 0
        else:
            self.counter += 1
            if self.verbose:
                print(f'EarlyStopping counter: {self.counter} out of {self.patience}')
            if self.counter >= self.patience:
                self.early_stop = True
        return self.early_stop

```

```

In [12]: # Model Evaluation
def evaluate_model(model, eval_loader, criterion):
    model.eval()
    eval_loss = 0.0

```

```

with torch.no_grad():
    for data, _ in eval_loader:
        data = data.to(device)
        output = model(data)

        loss = criterion(output, data)
        eval_loss += loss.item()

eval_loss /= len(eval_loader)
return eval_loss

# Model Training
def train_model(model, train_loader, valid_loader, criterion, optimizer):
    history = {
        'train_loss': [], 'val_loss': [],
    }

    for epoch in range(epochs):
        # Training phase
        model.train()
        train_loss = 0.0

        for batch_idx, (data, _) in enumerate(train_loader):
            data = data.to(device)

            optimizer.zero_grad()
            output = model(data)

            loss = criterion(output, data)
            loss.backward()
            optimizer.step()
            train_loss += loss.item()

        train_loss /= len(train_loader)

        # Validation phase
        val_loss = evaluate_model(model, valid_loader, criterion)

        # Store history
        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)

        msg = f' Train Loss: {train_loss:.6f}   Val Loss: {val_loss:.6f}'

        print(f'Epoch {epoch+1}/{epochs}: ', msg)

        # Check early stopping
        early_stopping(val_loss)
        if early_stopping.early_stop:
            print("Early stopping triggered")
            break

```



```

    else:
        torch.save(model.state_dict(), 'best_model.pth')

    # Load best model
    model.eval()
    model.load_state_dict(torch.load('best_model.pth'))
    # For the entire model
    for param in model.parameters():
        param.requires_grad = False
    return history, model

```

- Encoder and decoder subnetworks

```

In [13]: # Initialize random seed
torch.manual_seed(4487)
random.seed(4487)
np.random.seed(4487)
if torch.cuda.is_available():
    torch.cuda.manual_seed(4487)
    torch.backends.cudnn.deterministic = True

# Create model instance
model = DenseAE()

```

- Full auto-encoder network
 - Composed of the encoder and decoder models

```

In [14]: # Print model architecture
print(model)
summarize_layer_parameters(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

```

```
DenseAE(
  (encoder): Sequential(
    (0): Linear(in_features=784, out_features=10, bias=True)
    (1): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=10, out_features=784, bias=True)
    (1): Sigmoid()
  )
)
```

Module Name	Parameters
encoder.0.weight	7840
encoder.0.bias	10
decoder.0.weight	7840
decoder.0.bias	784

Total Trainable Parameters: 16474

Total Un-Trainable Parameters: 0

Using device: cpu

- compile the model
 - outputs are in [0,1], so use cross-entropy
- fit the model
 - the target is the same as the input

```
In [15]: # Prepare data & Create Dataloader
batch_size = 50
train_dataset = TensorDataset(vtrainI.flatten(1), vtrainY)
valid_dataset = TensorDataset(validI.flatten(1), validY)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)

# Compile the network (equivalent in PyTorch)
criterion = nn.BCELoss() # Binary Cross Entropy
optimizer = optim.SGD(model.parameters(), lr=0.15, momentum=0.9, nesterov=True)
early_stopping = EarlyStopping(min_delta=0.0001, patience=5)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion, optimizer, early_stopping)
```

```
Epoch 1/100: Train Loss: 0.637912 Val Loss: 0.469814
Epoch 2/100: Train Loss: 0.303034 Val Loss: 0.268170
Epoch 3/100: Train Loss: 0.261933 Val Loss: 0.262844
Epoch 4/100: Train Loss: 0.258127 Val Loss: 0.260269
```

Epoch 5/100:	Train Loss: 0.255428	Val Loss: 0.257824
Epoch 6/100:	Train Loss: 0.252544	Val Loss: 0.254862
Epoch 7/100:	Train Loss: 0.249084	Val Loss: 0.251481
Epoch 8/100:	Train Loss: 0.244960	Val Loss: 0.247262
Epoch 9/100:	Train Loss: 0.240220	Val Loss: 0.242610
Epoch 10/100:	Train Loss: 0.235373	Val Loss: 0.238270
Epoch 11/100:	Train Loss: 0.230822	Val Loss: 0.234054
Epoch 12/100:	Train Loss: 0.226659	Val Loss: 0.230240
Epoch 13/100:	Train Loss: 0.222906	Val Loss: 0.226922
Epoch 14/100:	Train Loss: 0.219542	Val Loss: 0.223722
Epoch 15/100:	Train Loss: 0.216449	Val Loss: 0.220806
Epoch 16/100:	Train Loss: 0.213627	Val Loss: 0.218259
Epoch 17/100:	Train Loss: 0.210993	Val Loss: 0.215763
Epoch 18/100:	Train Loss: 0.208527	Val Loss: 0.213323
Epoch 19/100:	Train Loss: 0.206214	Val Loss: 0.211159
Epoch 20/100:	Train Loss: 0.204110	Val Loss: 0.209132
Epoch 21/100:	Train Loss: 0.202167	Val Loss: 0.207212
Epoch 22/100:	Train Loss: 0.200323	Val Loss: 0.205472
Epoch 23/100:	Train Loss: 0.198663	Val Loss: 0.203854
Epoch 24/100:	Train Loss: 0.197131	Val Loss: 0.202389
Epoch 25/100:	Train Loss: 0.195723	Val Loss: 0.201129
Epoch 26/100:	Train Loss: 0.194420	Val Loss: 0.199945
Epoch 27/100:	Train Loss: 0.193260	Val Loss: 0.198795
Epoch 28/100:	Train Loss: 0.192175	Val Loss: 0.197742
Epoch 29/100:	Train Loss: 0.191150	Val Loss: 0.196889
Epoch 30/100:	Train Loss: 0.190239	Val Loss: 0.195815
Epoch 31/100:	Train Loss: 0.189341	Val Loss: 0.195091
Epoch 32/100:	Train Loss: 0.188510	Val Loss: 0.194292
Epoch 33/100:	Train Loss: 0.187721	Val Loss: 0.193378
Epoch 34/100:	Train Loss: 0.186933	Val Loss: 0.192681
Epoch 35/100:	Train Loss: 0.186271	Val Loss: 0.191972
Epoch 36/100:	Train Loss: 0.185586	Val Loss: 0.191270
Epoch 37/100:	Train Loss: 0.184943	Val Loss: 0.190670
Epoch 38/100:	Train Loss: 0.184364	Val Loss: 0.190205
Epoch 39/100:	Train Loss: 0.183779	Val Loss: 0.189548
Epoch 40/100:	Train Loss: 0.183226	Val Loss: 0.188965
Epoch 41/100:	Train Loss: 0.182662	Val Loss: 0.188454
Epoch 42/100:	Train Loss: 0.182106	Val Loss: 0.187820
Epoch 43/100:	Train Loss: 0.181516	Val Loss: 0.187153
Epoch 44/100:	Train Loss: 0.180925	Val Loss: 0.186763
Epoch 45/100:	Train Loss: 0.180349	Val Loss: 0.185916
Epoch 46/100:	Train Loss: 0.179686	Val Loss: 0.185227
Epoch 47/100:	Train Loss: 0.178985	Val Loss: 0.184548
Epoch 48/100:	Train Loss: 0.178314	Val Loss: 0.183848
Epoch 49/100:	Train Loss: 0.177599	Val Loss: 0.183079
Epoch 50/100:	Train Loss: 0.176896	Val Loss: 0.182414
Epoch 51/100:	Train Loss: 0.176239	Val Loss: 0.181788
Epoch 52/100:	Train Loss: 0.175585	Val Loss: 0.181123
Epoch 53/100:	Train Loss: 0.175002	Val Loss: 0.180450
Epoch 54/100:	Train Loss: 0.174424	Val Loss: 0.180004
Epoch 55/100:	Train Loss: 0.173923	Val Loss: 0.179520
Epoch 56/100:	Train Loss: 0.173484	Val Loss: 0.178953

Epoch 57/100:	Train Loss: 0.173059	Val Loss: 0.178648
Epoch 58/100:	Train Loss: 0.172704	Val Loss: 0.178299
Epoch 59/100:	Train Loss: 0.172393	Val Loss: 0.177951
Epoch 60/100:	Train Loss: 0.172109	Val Loss: 0.177603
Epoch 61/100:	Train Loss: 0.171841	Val Loss: 0.177365
Epoch 62/100:	Train Loss: 0.171615	Val Loss: 0.177145
Epoch 63/100:	Train Loss: 0.171369	Val Loss: 0.177008
Epoch 64/100:	Train Loss: 0.171219	Val Loss: 0.176874
Epoch 65/100:	Train Loss: 0.171039	Val Loss: 0.176641
Epoch 66/100:	Train Loss: 0.170885	Val Loss: 0.176642
EarlyStopping counter: 1 out of 5		
Epoch 67/100:	Train Loss: 0.170726	Val Loss: 0.176345
Epoch 68/100:	Train Loss: 0.170578	Val Loss: 0.176349
EarlyStopping counter: 1 out of 5		
Epoch 69/100:	Train Loss: 0.170462	Val Loss: 0.176123
Epoch 70/100:	Train Loss: 0.170331	Val Loss: 0.175927
Epoch 71/100:	Train Loss: 0.170219	Val Loss: 0.175871
EarlyStopping counter: 1 out of 5		
Epoch 72/100:	Train Loss: 0.170115	Val Loss: 0.175927
EarlyStopping counter: 2 out of 5		
Epoch 73/100:	Train Loss: 0.170019	Val Loss: 0.175686
Epoch 74/100:	Train Loss: 0.169945	Val Loss: 0.175632
EarlyStopping counter: 1 out of 5		
Epoch 75/100:	Train Loss: 0.169867	Val Loss: 0.175490
Epoch 76/100:	Train Loss: 0.169779	Val Loss: 0.175440
EarlyStopping counter: 1 out of 5		
Epoch 77/100:	Train Loss: 0.169688	Val Loss: 0.175444
EarlyStopping counter: 2 out of 5		
Epoch 78/100:	Train Loss: 0.169630	Val Loss: 0.175373
Epoch 79/100:	Train Loss: 0.169548	Val Loss: 0.175149
Epoch 80/100:	Train Loss: 0.169460	Val Loss: 0.175118
EarlyStopping counter: 1 out of 5		
Epoch 81/100:	Train Loss: 0.169409	Val Loss: 0.175032
Epoch 82/100:	Train Loss: 0.169335	Val Loss: 0.174981
EarlyStopping counter: 1 out of 5		
Epoch 83/100:	Train Loss: 0.169269	Val Loss: 0.174923
Epoch 84/100:	Train Loss: 0.169217	Val Loss: 0.174995
EarlyStopping counter: 1 out of 5		
Epoch 85/100:	Train Loss: 0.169168	Val Loss: 0.174824
EarlyStopping counter: 2 out of 5		
Epoch 86/100:	Train Loss: 0.169130	Val Loss: 0.174692
Epoch 87/100:	Train Loss: 0.169057	Val Loss: 0.174706
EarlyStopping counter: 1 out of 5		
Epoch 88/100:	Train Loss: 0.169015	Val Loss: 0.174798
EarlyStopping counter: 2 out of 5		
Epoch 89/100:	Train Loss: 0.168963	Val Loss: 0.174667
EarlyStopping counter: 3 out of 5		
Epoch 90/100:	Train Loss: 0.168910	Val Loss: 0.174526
Epoch 91/100:	Train Loss: 0.168880	Val Loss: 0.174630
EarlyStopping counter: 1 out of 5		
Epoch 92/100:	Train Loss: 0.168832	Val Loss: 0.174552
EarlyStopping counter: 2 out of 5		

```

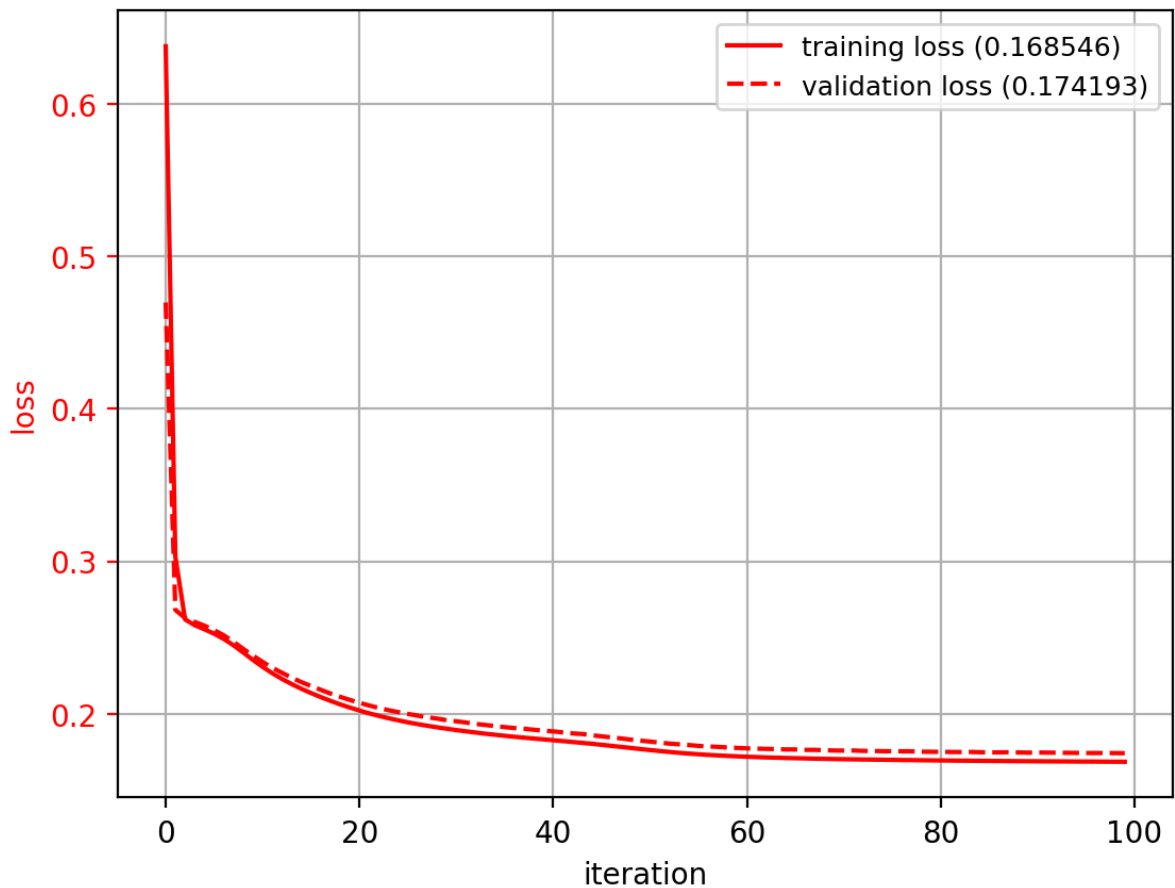
Epoch 93/100:   Train Loss: 0.168791   Val Loss: 0.174489
EarlyStopping counter: 3 out of 5
Epoch 94/100:   Train Loss: 0.168759   Val Loss: 0.174410
Epoch 95/100:   Train Loss: 0.168692   Val Loss: 0.174363
EarlyStopping counter: 1 out of 5
Epoch 96/100:   Train Loss: 0.168678   Val Loss: 0.174292
Epoch 97/100:   Train Loss: 0.168651   Val Loss: 0.174281
EarlyStopping counter: 1 out of 5
Epoch 98/100:   Train Loss: 0.168587   Val Loss: 0.174306
EarlyStopping counter: 2 out of 5
Epoch 99/100:   Train Loss: 0.168557   Val Loss: 0.174218
EarlyStopping counter: 3 out of 5
Epoch 100/100:  Train Loss: 0.168546   Val Loss: 0.174193
EarlyStopping counter: 4 out of 5

```

```

In [16]: # Plot the training history
plot_history(history)

```

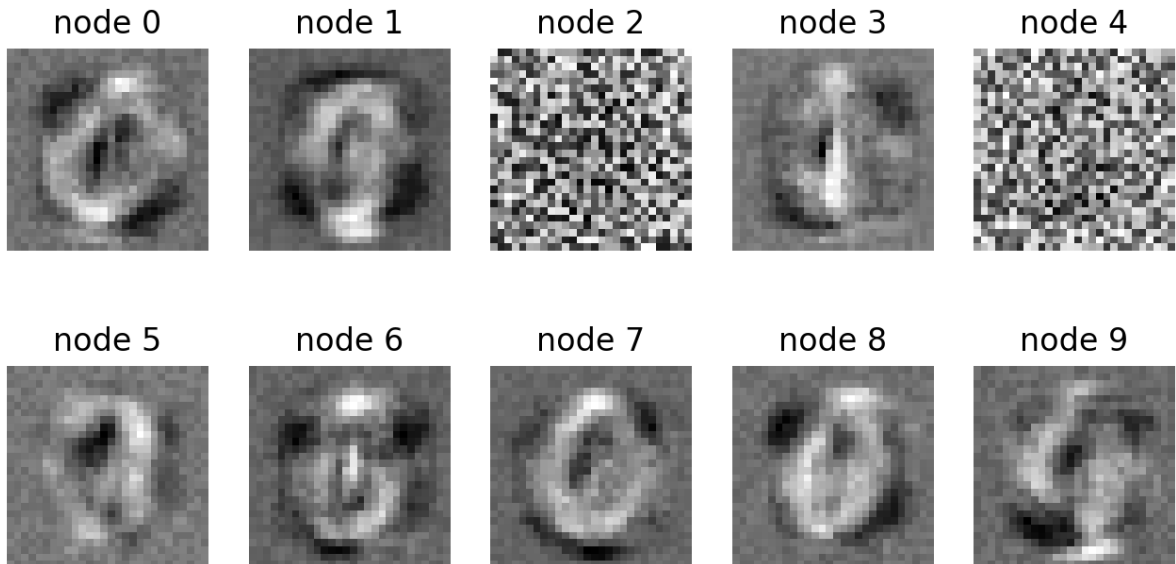


- Visualize the weights of the hidden layer in E
 - each hidden node activates on a particular structure

```

In [17]: W = model.encoder[0].weight.cpu().numpy()
filter_list = [W[i].reshape((28,28)) for i in range(W.shape[0])]
plt.figure(figsize=(8,4))
show_imgs(filter_list, nc=5, titles="node %d")

```



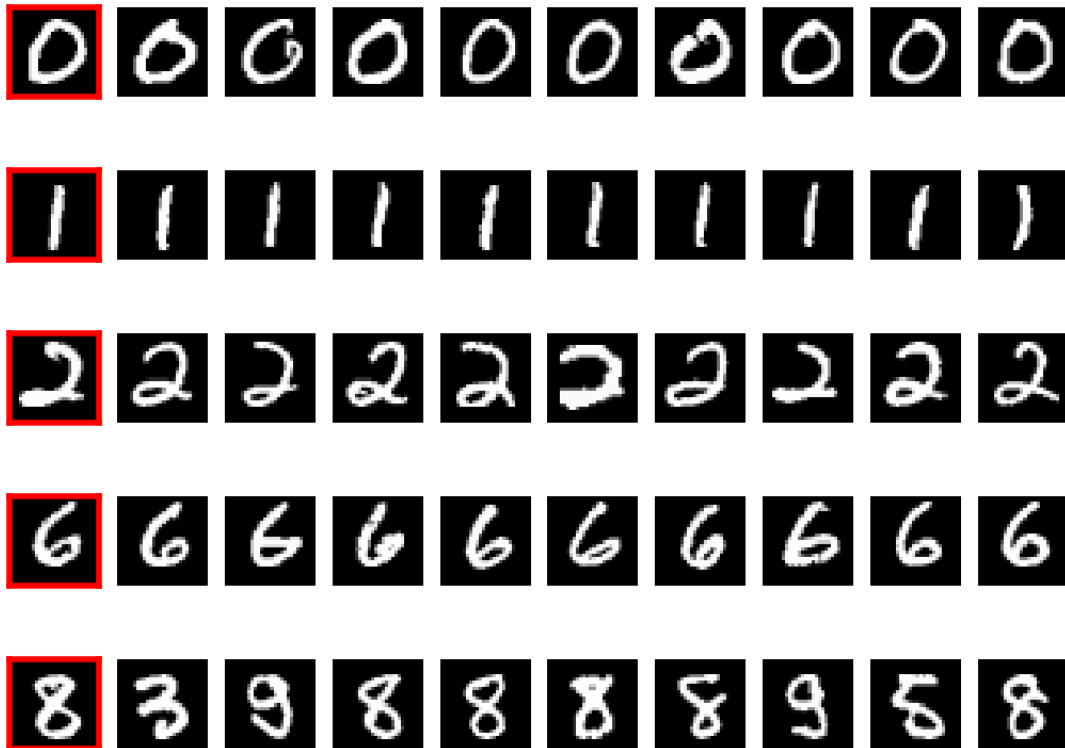
- Encode images into low-dim representation.

```
In [18]: model.eval()
Z = model.encoder(trainI.flatten(1)).cpu().numpy()
Z.shape
```

```
Out[18]: (6000, 10)
```

```
In [19]: Wlist = []
for ii in [210,4,101,9,294]:
    d = metrics.pairwise.euclidean_distances(Z, [Z[ii]])
    inds = np.argsort(d.ravel())
    for x in inds[0:10]:
        Wlist.append(trainI[x].numpy().reshape((28,28))*255.0)

Zfig = plt.figure()
tmp = [1,0,0,0,0,0,0,0,0,0]*5
tmp2 = [0]*50
show_imgs(Wlist, nc=10, highlight_red=tmp, highlight_green=tmp2)
plt.close()
```

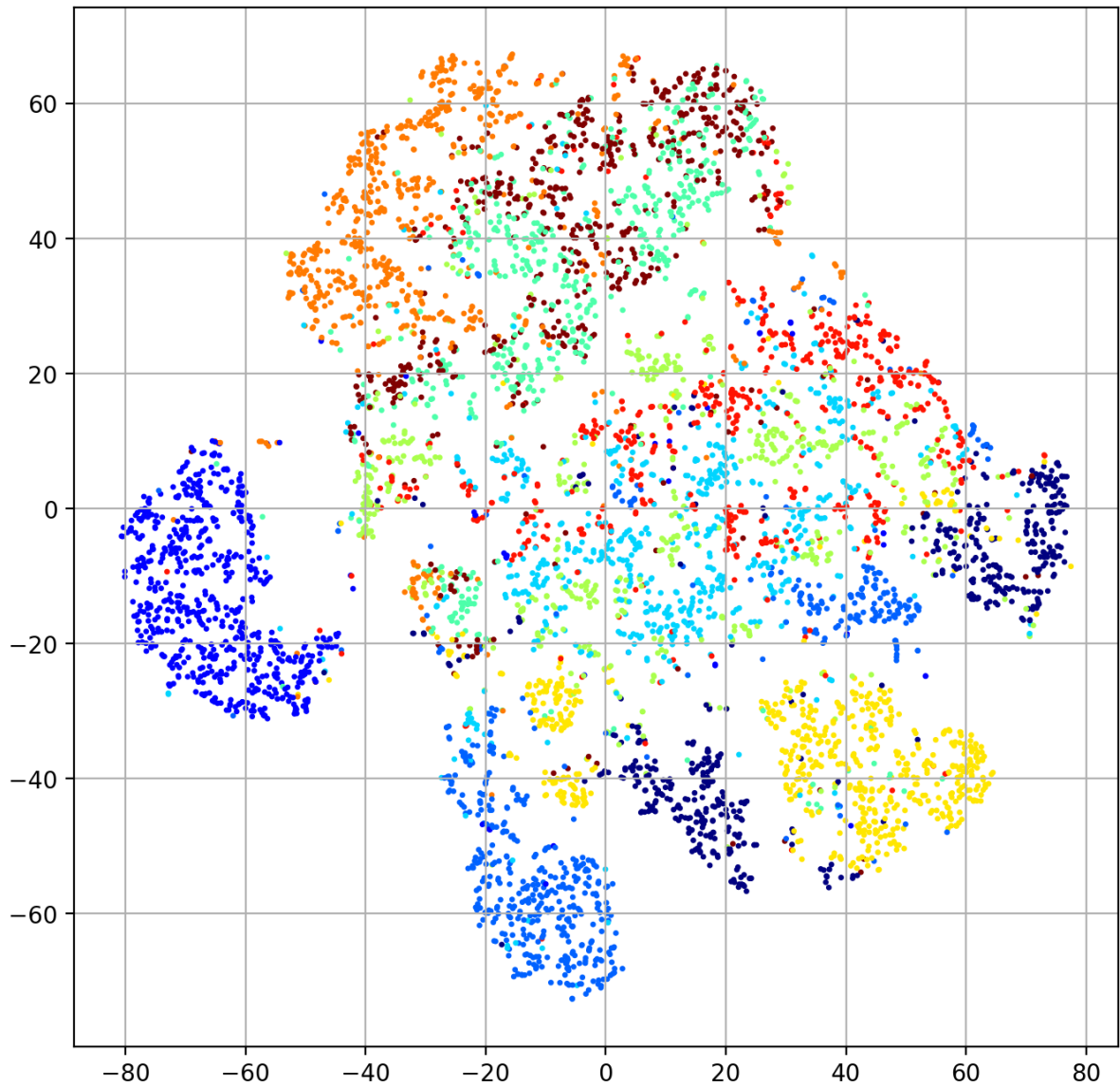


- for the given highlighted red digit
 - show the 9 nearest neighbors in the latent space

```
In [20]: def plot_embedding(X, Y, perplexity):
#IPython.core.display.set_matplotlib_formats("retina") # switch to
tsne = manifold.TSNE(n_components=2, init='pca', perplexity=perple
x_pts = tsne.fit_transform(X)
fig = plt.figure(figsize=(8,8))
plt.scatter(x_pts[:, 0], x_pts[:, 1], c=Y, s=2, cmap=plt.get_cmap(
plt.grid()
plt.show()
plt.close()
#IPython.core.display.set_matplotlib_formats("svg")
```

- Visualize a t-SNE embedding of the latent-space

```
In [21]: plot_embedding(Z, trainY, perplexity=30.)
```



- Visualize the reconstruction of the input image

```
In [22]: model.eval()
testXrecon = model(testI.flatten(1).to(device))
```

```
In [23]: imglist = []
for j,i in enumerate(range(0,100,10)):
    tmp = np.hstack( (testI[i].reshape((28,28))*255.0, testXrecon[i].r
    imglist.append(tmp)
rfig = plt.figure(figsize=(10,3))
show_imgs(imglist,nc=5)
plt.close()
```




Better Representations

- How to encourage better representations in the latent space?
- Two approaches:
 1. Regularize the latent space
 - e.g., using a sparsity-inducing regularizer of the latent vector
 - $L = ||\mathbf{x} - g(f(\mathbf{x}))||^2 + \lambda ||f(\mathbf{x})||_1$
 2. Denoising auto-encoder (DAE)
 - add noise to make the encoder learn more about the data manifold.

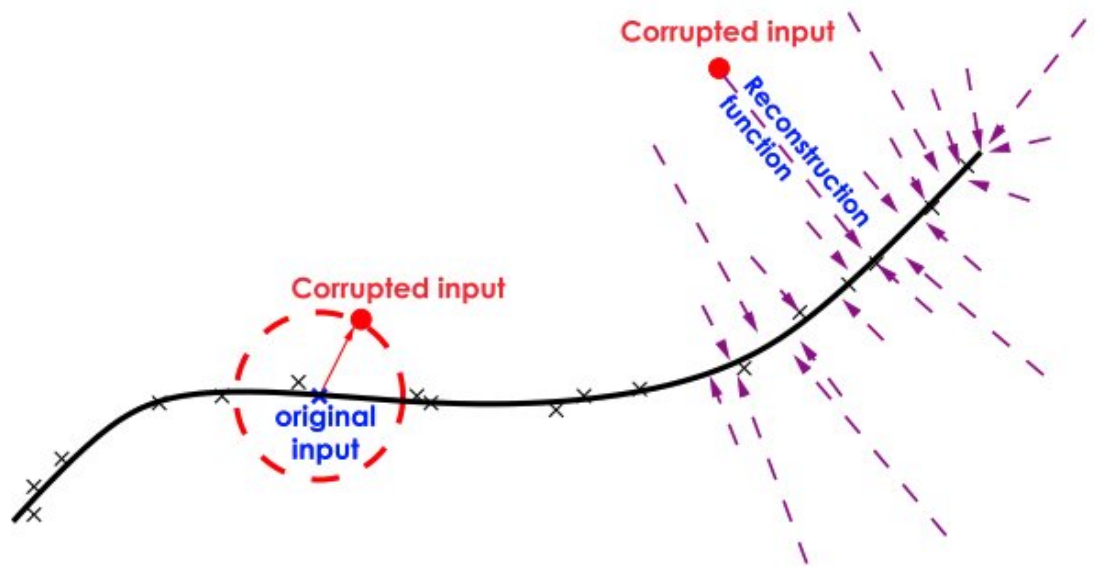
Denoising Autoencoder

- **Idea:** the AE's goal is to minimize $||\mathbf{x} - g(f(\mathbf{x}))||^2$.
 - the minimum occurs when $g(f(\mathbf{x})) = \mathbf{x}$, i.e., the identity function.
- **Problem:** The network can obtain this arbitrarily if enough capacity.
 - but without learning anything about the data distribution.
- **Solution:**
 - Force the AE to learn more about the data by corrupting the inputs.
 - minimize $||\mathbf{x} - g(f(\tilde{\mathbf{x}}))||^2$
 - $\tilde{\mathbf{x}}$ is a corrupted input
 - e.g., adding Gaussian noise, or zeroing some values
 - AE will learn how to "undo" the corruption, which requires learning the structure of the data manifold.
 - allows "code" size to be larger than the input size

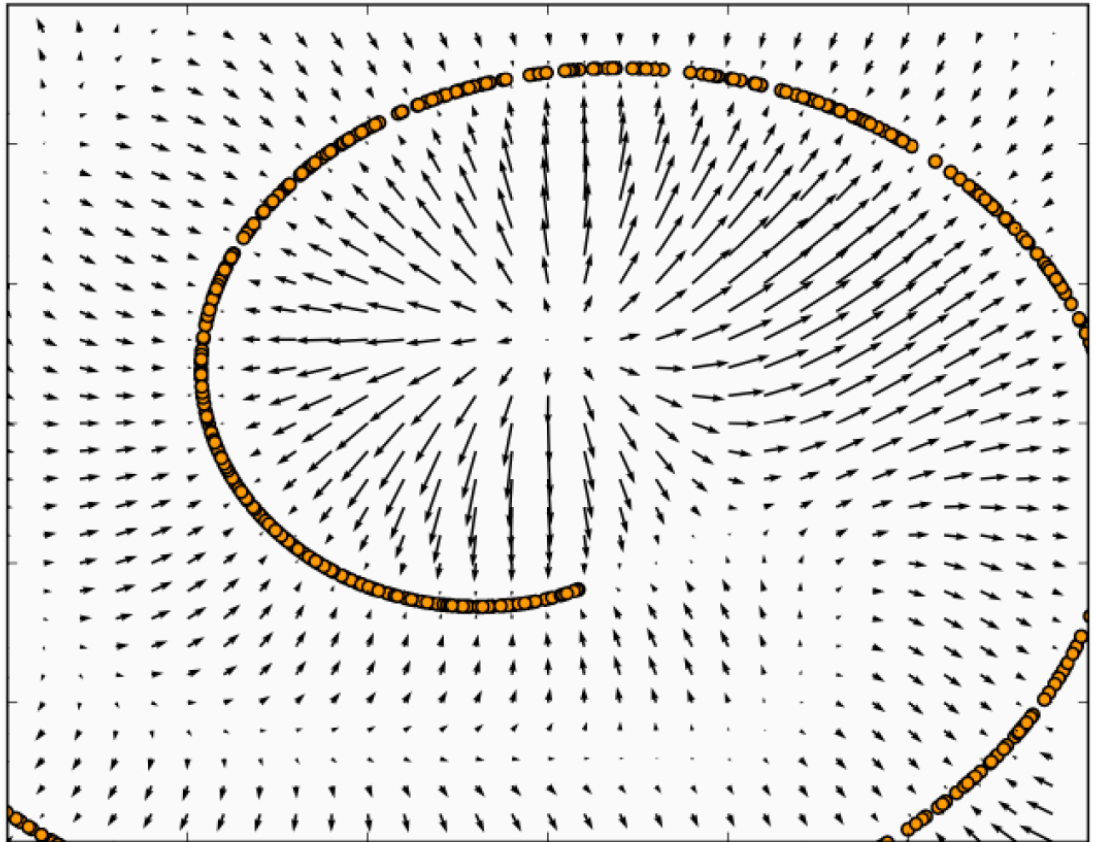
DAE Interpretation (1)

- DAE is learning a vector field from the corrupted $\tilde{\mathbf{x}}$ to the clean $\mathbf{x}$ (the data)

manifold).

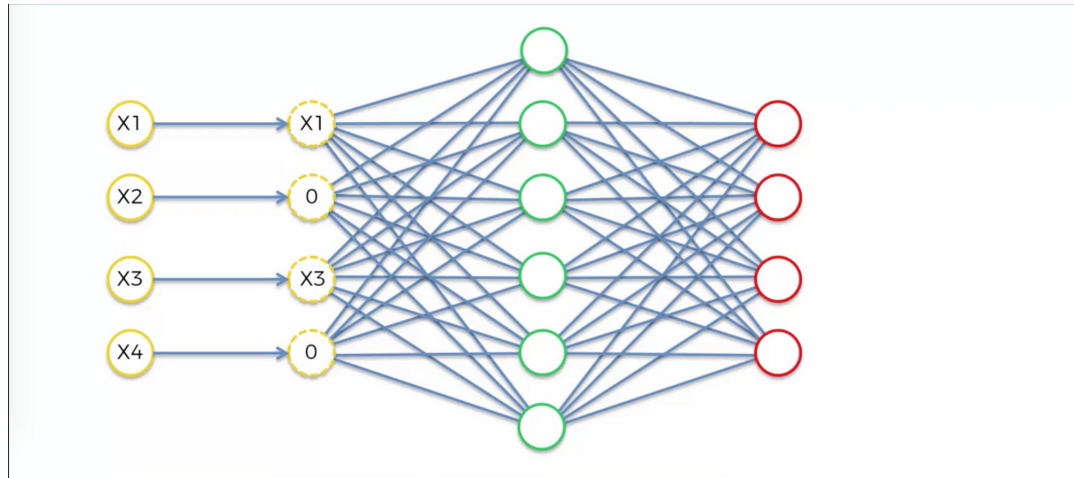


Example



Denoising Autoencoder

- randomly corrupt the input (by setting values to 0)
- implemented by applying Dropout on the inputs.



```
In [24]: # Define the Denoising Autoencoder
class DenoiseAE(DenseAE):
    def __init__(self):
        super(DenoiseAE, self).__init__()
        # Encoder
        self.dropout = nn.Dropout1d(p=0.3)

    def forward(self, x):
        encoded = self.encoder(self.dropout(x))
        decoded = self.decoder(encoded)
        return decoded

    def encode_proc(self, x):
        return self.encoder(self.dropout(x))
```

- Encoder and decoder subnetworks

```
In [25]: # Initialize random seed
torch.manual_seed(4487)
random.seed(4487)
np.random.seed(4487)
if torch.cuda.is_available():
    torch.cuda.manual_seed(4487)
    torch.backends.cudnn.deterministic = True
```

```
In [26]: # Create model instance
model = DenoiseAE()

# Print model architecture
print(model)
summarize_layer_parameters(model)

# Move model to device (GPU if available)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")
```

```
DenoiseAE(
  (encoder): Sequential(
    (0): Linear(in_features=784, out_features=10, bias=True)
    (1): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=10, out_features=784, bias=True)
    (1): Sigmoid()
  )
  (dropout): Dropout1d(p=0.3, inplace=False)
)
```

Module Name	Parameters
encoder.0.weight	7840
encoder.0.bias	10
decoder.0.weight	7840
decoder.0.bias	784

Total Trainable Parameters: 16474

Total Un-Trainable Parameters: 0

Using device: cpu

- Full auto-encoder network
 - Composed of the encoder and decoder models
- Compile the model

```
In [27]: # Compile the network (equivalent in PyTorch)
optimizer = optim.SGD(model.parameters(), lr=0.15, momentum=0.9, nesterov=True)
early_stopping = EarlyStopping(min_delta=0.0001, patience=5)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion)
```

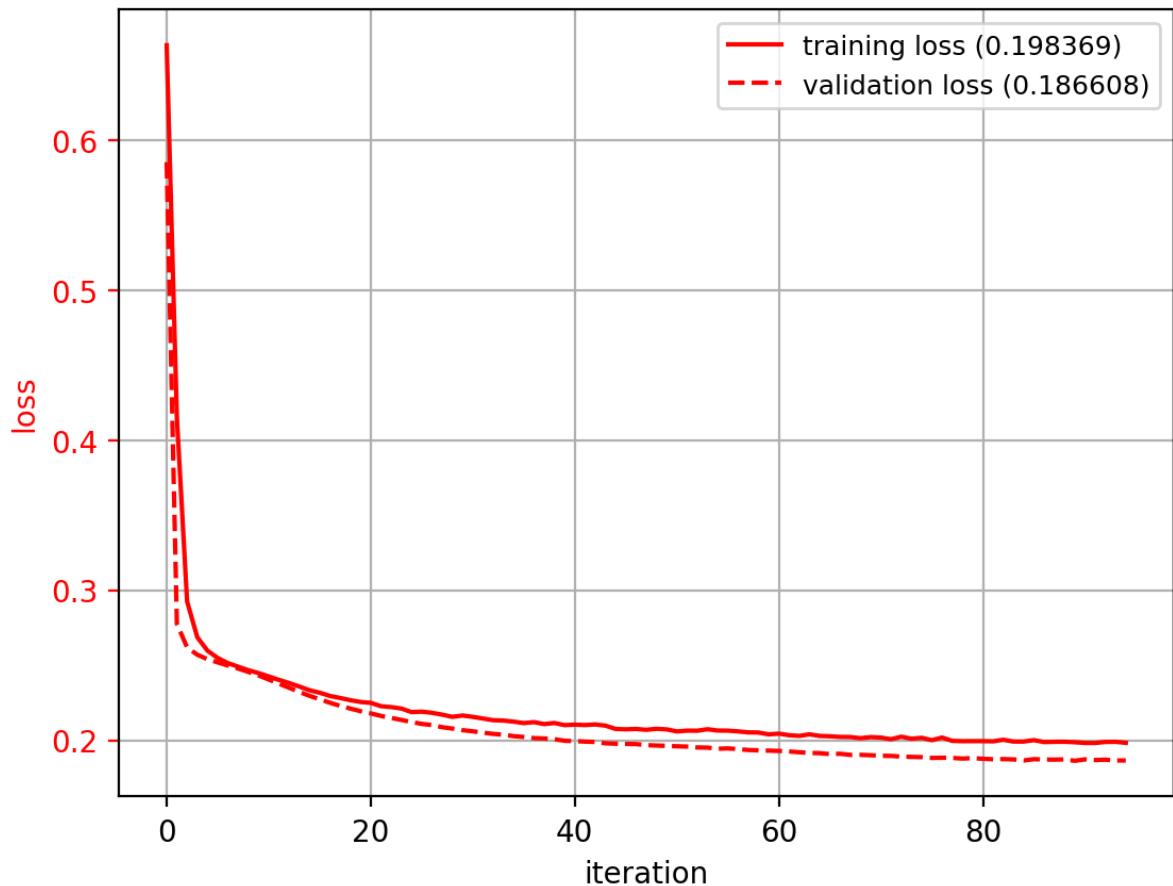
```
Epoch 1/100:  Train Loss: 0.663139    Val Loss: 0.585272
Epoch 2/100:  Train Loss: 0.415039    Val Loss: 0.277693
Epoch 3/100:  Train Loss: 0.292573    Val Loss: 0.261932
Epoch 4/100:  Train Loss: 0.268711    Val Loss: 0.257069
Epoch 5/100:  Train Loss: 0.259757    Val Loss: 0.254075
Epoch 6/100:  Train Loss: 0.254888    Val Loss: 0.251974
Epoch 7/100:  Train Loss: 0.251477    Val Loss: 0.249859
Epoch 8/100:  Train Loss: 0.249086    Val Loss: 0.247837
Epoch 9/100:  Train Loss: 0.246774    Val Loss: 0.245613
Epoch 10/100: Train Loss: 0.244812    Val Loss: 0.243266
```

Epoch 11/100:	Train Loss: 0.242597	Val Loss: 0.240533
Epoch 12/100:	Train Loss: 0.240303	Val Loss: 0.237757
Epoch 13/100:	Train Loss: 0.238199	Val Loss: 0.235045
Epoch 14/100:	Train Loss: 0.235877	Val Loss: 0.232279
Epoch 15/100:	Train Loss: 0.233393	Val Loss: 0.229648
Epoch 16/100:	Train Loss: 0.231749	Val Loss: 0.227419
Epoch 17/100:	Train Loss: 0.229605	Val Loss: 0.225069
Epoch 18/100:	Train Loss: 0.228243	Val Loss: 0.223005
Epoch 19/100:	Train Loss: 0.226779	Val Loss: 0.221174
Epoch 20/100:	Train Loss: 0.225594	Val Loss: 0.219455
Epoch 21/100:	Train Loss: 0.224995	Val Loss: 0.217946
Epoch 22/100:	Train Loss: 0.222780	Val Loss: 0.216303
Epoch 23/100:	Train Loss: 0.222167	Val Loss: 0.214946
Epoch 24/100:	Train Loss: 0.221213	Val Loss: 0.213624
Epoch 25/100:	Train Loss: 0.218857	Val Loss: 0.212228
Epoch 26/100:	Train Loss: 0.219132	Val Loss: 0.210964
Epoch 27/100:	Train Loss: 0.218343	Val Loss: 0.210198
Epoch 28/100:	Train Loss: 0.217137	Val Loss: 0.208778
Epoch 29/100:	Train Loss: 0.215684	Val Loss: 0.207874
Epoch 30/100:	Train Loss: 0.216563	Val Loss: 0.206786
Epoch 31/100:	Train Loss: 0.215678	Val Loss: 0.206049
Epoch 32/100:	Train Loss: 0.214555	Val Loss: 0.205136
Epoch 33/100:	Train Loss: 0.213432	Val Loss: 0.204313
Epoch 34/100:	Train Loss: 0.213197	Val Loss: 0.203789
Epoch 35/100:	Train Loss: 0.212485	Val Loss: 0.202811
Epoch 36/100:	Train Loss: 0.211512	Val Loss: 0.202271
Epoch 37/100:	Train Loss: 0.212143	Val Loss: 0.201497
Epoch 38/100:	Train Loss: 0.210849	Val Loss: 0.201238
Epoch 39/100:	Train Loss: 0.211571	Val Loss: 0.200819
Epoch 40/100:	Train Loss: 0.210166	Val Loss: 0.199723
Epoch 41/100:	Train Loss: 0.210479	Val Loss: 0.199497
Epoch 42/100:	Train Loss: 0.210196	Val Loss: 0.199164
Epoch 43/100:	Train Loss: 0.210616	Val Loss: 0.198921
Epoch 44/100:	Train Loss: 0.209822	Val Loss: 0.198044
Epoch 45/100:	Train Loss: 0.207615	Val Loss: 0.197781
Epoch 46/100:	Train Loss: 0.207357	Val Loss: 0.197627
Epoch 47/100:	Train Loss: 0.207621	Val Loss: 0.197535
EarlyStopping counter: 1 out of 5		
Epoch 48/100:	Train Loss: 0.207042	Val Loss: 0.196824
Epoch 49/100:	Train Loss: 0.207717	Val Loss: 0.196579
Epoch 50/100:	Train Loss: 0.207334	Val Loss: 0.196344
Epoch 51/100:	Train Loss: 0.205966	Val Loss: 0.196063
Epoch 52/100:	Train Loss: 0.206493	Val Loss: 0.195844
Epoch 53/100:	Train Loss: 0.206423	Val Loss: 0.195273
Epoch 54/100:	Train Loss: 0.207422	Val Loss: 0.195163
Epoch 55/100:	Train Loss: 0.206548	Val Loss: 0.194415
Epoch 56/100:	Train Loss: 0.206487	Val Loss: 0.194649
EarlyStopping counter: 1 out of 5		
Epoch 57/100:	Train Loss: 0.205990	Val Loss: 0.194150
Epoch 58/100:	Train Loss: 0.205330	Val Loss: 0.193602
Epoch 59/100:	Train Loss: 0.205233	Val Loss: 0.193343
Epoch 60/100:	Train Loss: 0.203984	Val Loss: 0.193162

Epoch 61/100:	Train Loss: 0.204487	Val Loss: 0.192963
Epoch 62/100:	Train Loss: 0.203444	Val Loss: 0.192633
Epoch 63/100:	Train Loss: 0.202965	Val Loss: 0.192073
Epoch 64/100:	Train Loss: 0.204029	Val Loss: 0.191620
Epoch 65/100:	Train Loss: 0.202962	Val Loss: 0.191495
Epoch 66/100:	Train Loss: 0.202726	Val Loss: 0.190814
Epoch 67/100:	Train Loss: 0.202297	Val Loss: 0.191080
EarlyStopping counter: 1 out of 5		
Epoch 68/100:	Train Loss: 0.202235	Val Loss: 0.190337
Epoch 69/100:	Train Loss: 0.201454	Val Loss: 0.190353
EarlyStopping counter: 1 out of 5		
Epoch 70/100:	Train Loss: 0.202176	Val Loss: 0.190100
Epoch 71/100:	Train Loss: 0.201855	Val Loss: 0.189776
Epoch 72/100:	Train Loss: 0.200750	Val Loss: 0.189641
Epoch 73/100:	Train Loss: 0.202449	Val Loss: 0.189179
Epoch 74/100:	Train Loss: 0.201106	Val Loss: 0.188996
Epoch 75/100:	Train Loss: 0.201692	Val Loss: 0.188819
Epoch 76/100:	Train Loss: 0.200117	Val Loss: 0.188312
Epoch 77/100:	Train Loss: 0.201888	Val Loss: 0.188415
EarlyStopping counter: 1 out of 5		
Epoch 78/100:	Train Loss: 0.199655	Val Loss: 0.188353
EarlyStopping counter: 2 out of 5		
Epoch 79/100:	Train Loss: 0.199468	Val Loss: 0.187858
Epoch 80/100:	Train Loss: 0.199496	Val Loss: 0.188183
EarlyStopping counter: 1 out of 5		
Epoch 81/100:	Train Loss: 0.199492	Val Loss: 0.187725
Epoch 82/100:	Train Loss: 0.199344	Val Loss: 0.187424
Epoch 83/100:	Train Loss: 0.200420	Val Loss: 0.187538
EarlyStopping counter: 1 out of 5		
Epoch 84/100:	Train Loss: 0.199193	Val Loss: 0.187318
Epoch 85/100:	Train Loss: 0.199204	Val Loss: 0.186612
Epoch 86/100:	Train Loss: 0.200140	Val Loss: 0.187475
EarlyStopping counter: 1 out of 5		
Epoch 87/100:	Train Loss: 0.198882	Val Loss: 0.187252
EarlyStopping counter: 2 out of 5		
Epoch 88/100:	Train Loss: 0.199043	Val Loss: 0.187174
EarlyStopping counter: 3 out of 5		
Epoch 89/100:	Train Loss: 0.199083	Val Loss: 0.187336
EarlyStopping counter: 4 out of 5		
Epoch 90/100:	Train Loss: 0.198756	Val Loss: 0.186408
Epoch 91/100:	Train Loss: 0.198245	Val Loss: 0.187342
EarlyStopping counter: 1 out of 5		
Epoch 92/100:	Train Loss: 0.198248	Val Loss: 0.186890
EarlyStopping counter: 2 out of 5		
Epoch 93/100:	Train Loss: 0.198947	Val Loss: 0.187134
EarlyStopping counter: 3 out of 5		
Epoch 94/100:	Train Loss: 0.199036	Val Loss: 0.186597
EarlyStopping counter: 4 out of 5		
Epoch 95/100:	Train Loss: 0.198369	Val Loss: 0.186608
EarlyStopping counter: 5 out of 5		
Early stopping triggered		

- Fit the model

```
In [28]: plot_history(history)
```



- Encode images into low-dim representation.
 - Note: at test time, Dropout layers do not add noise.

```
In [29]: model.eval()
Z = model.encoder(trainI.flatten(1)).cpu().numpy()
Z.shape
```

```
Out[29]: (6000, 10)
```

- Visualize the nearby neighbors in the low-dim representation.
 - each row represents one set of neighbors

```
In [30]: Wlist = []
for ii in [210,4,101,9,294]:
    d = metrics.pairwise.euclidean_distances(Z, [Z[ii]])
    inds = np.argsort(d.ravel())
    for x in inds[0:10]:
        Wlist.append(trainI[x].numpy().reshape((28,28))*255.0)
```

```

tmp = [1,0,0,0,0,0,0,0,0,0]*5
tmp2 = [0]*50
Zfig = plt.figure()
show_imgs(Wlist, nc=10, highlight_red=tmp, highlight_green=tmp2)
plt.close()

```

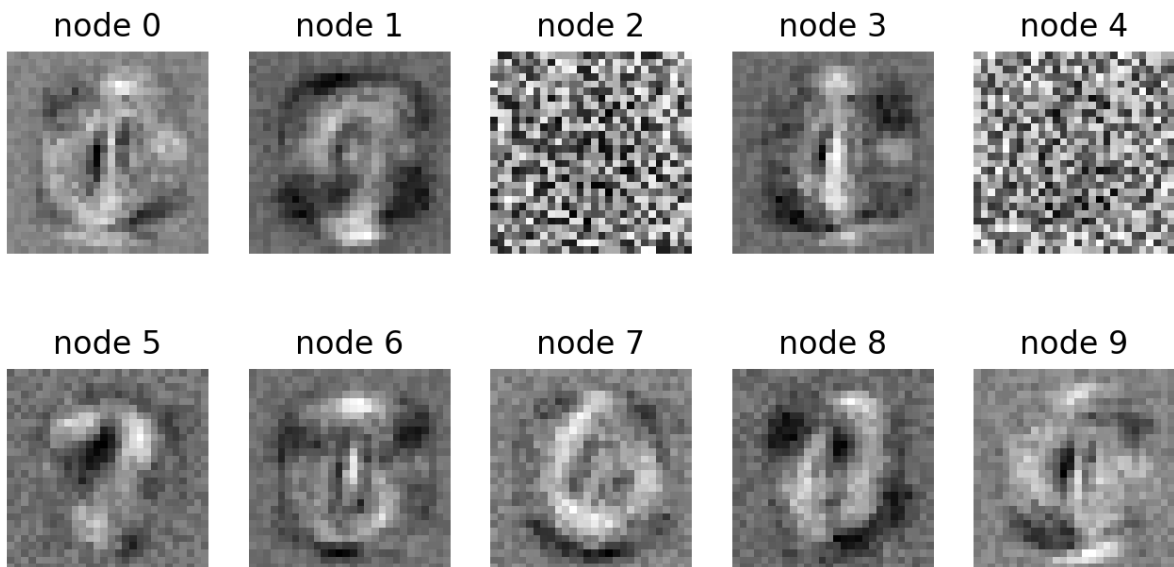


- Visualize the weights of the hidden layer that generate codes
 - each hidden node activates on a particular structure

```

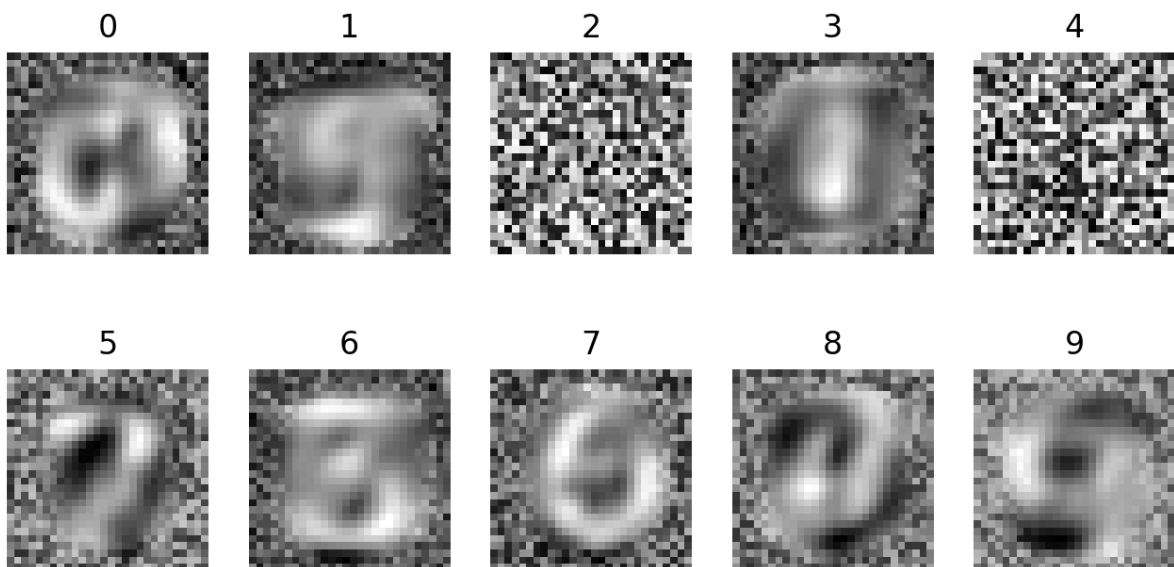
In [31]: W = model.encoder[0].weight.cpu().numpy()
filter_list = [W[i].reshape((28,28)) for i in range(W.shape[0])]
plt.figure(figsize=(8,4))
show_imgs(filter_list, nc=5, titles="node %d")

```

- Visualize the weights that project the code into an image
 - the image structures match those of the encoder, but are smoother.

```
In [32]: W = model.decoder[0].weight.T.cpu().numpy()
filter_list = [W[i,:].reshape((28,28)) for i in range(W.shape[0])]
plt.figure(figsize=(8,4))
show_imgs(filter_list, nc=5, titles="%d")
```



- Visualize the reconstruction of the input image

```
In [33]: model.eval()
testXrecon = model(testI.flatten(1).to(device))
```

```
In [34]: imglist = []
for j,i in enumerate(range(0,100,10)):
```

```

    tmp = np.hstack( (testI[i].reshape((28,28))*255.0, testXrecon[i].r
    imglist.append(tmp)
rfig = plt.figure(figsize=(10,3))
show_imgs(imglist,nc=5)
plt.close()

```



- Corrupt the input image and encode-decode
 - performs "denoising" of the input

```

In [35]: noisytest = testI * np.random.binomial(n=1,p=1-0.3,size=testI.shape)
testXrecon = model(noisytest.float().flatten(1).to(device))

```

```

/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_75225/260145
4893.py:1: DeprecationWarning: __array_wrap__ must accept context and r
eturn_scalar arguments (positionally) in the future. (Deprecated NumPy
2.0)
    noisytest = testI * np.random.binomial(n=1,p=1-0.3,size=testI.shape)

```

```

In [36]: imglist = []
for j,i in enumerate(range(0,100,10)):
    tmp = np.hstack( (noisytest[i].reshape((28,28)).numpy()*255.0, tes
    imglist.append(tmp)
rfig = plt.figure(figsize=(10,3))
show_imgs(imglist,nc=5)
plt.close()

```



DAE Interpretation 2

- Learning a reconstruction distribution $p_{\text{recon}}(\mathbf{x}|\tilde{\mathbf{x}})$

- Generative process:
 1. sample training example $\mathbf{x}$
 2. corrupt $\tilde{\mathbf{x}} \sim C(\tilde{\mathbf{x}}|\mathbf{x})$
 3. use $(\tilde{\mathbf{x}}, \mathbf{x})$ as a training example to estimate

$$p_{\text{recon}}(\mathbf{x}|\tilde{\mathbf{x}}) = p_{\text{decoder}}(\mathbf{x}|\mathbf{z}),$$

$$\mathbf{z} = f(\tilde{\mathbf{x}})$$

- Assume $p_{\text{decoder}}(\mathbf{x}|\mathbf{z})$ is Gaussian:
 - Note that $\log p_{\text{decoder}}(\mathbf{x}|\mathbf{z}) = -||\mathbf{x} - g(\mathbf{z})||^2$ (negative MSE loss)
- Minimize the loss over the training distribution
 - $\sum_i ||\mathbf{x}_i - g(f(\tilde{\mathbf{x}}_i))||^2$
- is equivalent to maximizing the expected log probability

$$\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} \mathbb{E}_{\tilde{\mathbf{x}} \sim C(\tilde{\mathbf{x}}|\mathbf{x})} [\log p_{\text{decoder}}(\mathbf{x}|\mathbf{z} = f(\tilde{\mathbf{x}}))]$$

- expectation is over the data and the corruption.

Convolutional Auto-Encoder

- Encoder - a standard CNN (w/o classifier)
 - Extract a feature map
- Decoder
 - the opposite architecture
 - Replace maxpooling with upsampling

```
In [37]: class ConvAE(nn.Module):
def __init__(self):
    super(ConvAE, self).__init__()

    self.encoder = nn.Sequential(
        nn.Conv2d(1, 16, kernel_size=3, stride=1, padding=1), # (
        nn.ReLU(True),
        nn.MaxPool2d(kernel_size=2, stride=2, padding=0),      # (
        nn.Conv2d(16, 8, kernel_size=3, stride=1, padding=1), # (
        nn.ReLU(True),
        nn.MaxPool2d(kernel_size=2, stride=2, padding=0),      # (
        nn.Conv2d(8, 8, kernel_size=3, stride=1, padding=1),   # (
        nn.ReLU(True),
        nn.MaxPool2d(kernel_size=2, stride=2, padding=1),      # (
```

```

    )

    self.decoder = nn.Sequential(
        nn.Conv2d(8, 8, kernel_size=3, stride=1, padding=1), # (
        nn.ReLU(True),
        nn.Upsample(scale_factor=2, mode='nearest'), # (

        nn.Conv2d(8, 8, kernel_size=3, stride=1, padding=1), # (
        nn.ReLU(True),
        nn.Upsample(scale_factor=2, mode='nearest'), # (

        # Note: Adjusted to match dimensions - no padding to reduce
        nn.Conv2d(8, 16, kernel_size=3, stride=1, padding=0), # (
        nn.ReLU(True),
        nn.Upsample(scale_factor=2, mode='nearest'), # (

        nn.Conv2d(16, 1, kernel_size=3, stride=1, padding=1), # (
        nn.Sigmoid()
    )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

```

- Encoder and Decoders

```

In [38]: # Create model instance
model = ConvAE()

# Print model architecture
print(model)
summarize_layer_parameters(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

```

```

ConvAE(
  (encoder): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

```

```

        (6): Conv2d(8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
        (7): ReLU(inplace=True)
        (8): MaxPool2d(kernel_size=2, stride=2, padding=1, dilation=1, ceil
_mode=False)
    )
    (decoder): Sequential(
      (0): Conv2d(8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (1): ReLU(inplace=True)
      (2): Upsample(scale_factor=2.0, mode='nearest')
      (3): Conv2d(8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (4): ReLU(inplace=True)
      (5): Upsample(scale_factor=2.0, mode='nearest')
      (6): Conv2d(8, 16, kernel_size=(3, 3), stride=(1, 1))
      (7): ReLU(inplace=True)
      (8): Upsample(scale_factor=2.0, mode='nearest')
      (9): Conv2d(16, 1, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
      (10): Sigmoid()
    )
  )

```

Module Name	Parameters
encoder.0.weight	144
encoder.0.bias	16
encoder.3.weight	1152
encoder.3.bias	8
encoder.6.weight	576
encoder.6.bias	8
decoder.0.weight	576
decoder.0.bias	8
decoder.3.weight	576
decoder.3.bias	8
decoder.6.weight	1152
decoder.6.bias	16
decoder.9.weight	144
decoder.9.bias	1

Total Trainable Parameters: 4385

Total Un-Trainable Parameters: 0

Using device: cpu

```

In [39]: # Prepare data & Create Dataloader
batch_size = 128
train_dataset = TensorDataset(vtrainI, vtrainY)
valid_dataset = TensorDataset(validI, validY)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffl

```

```

valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=True)

# Compile the network (equivalent in PyTorch)
criterion = nn.BCELoss() # Binary Cross Entropy
optimizer = optim.Adam(model.parameters())
early_stopping = EarlyStopping(min_delta=0.0001, patience=5)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion, optimizer, early_stopping)

```

```

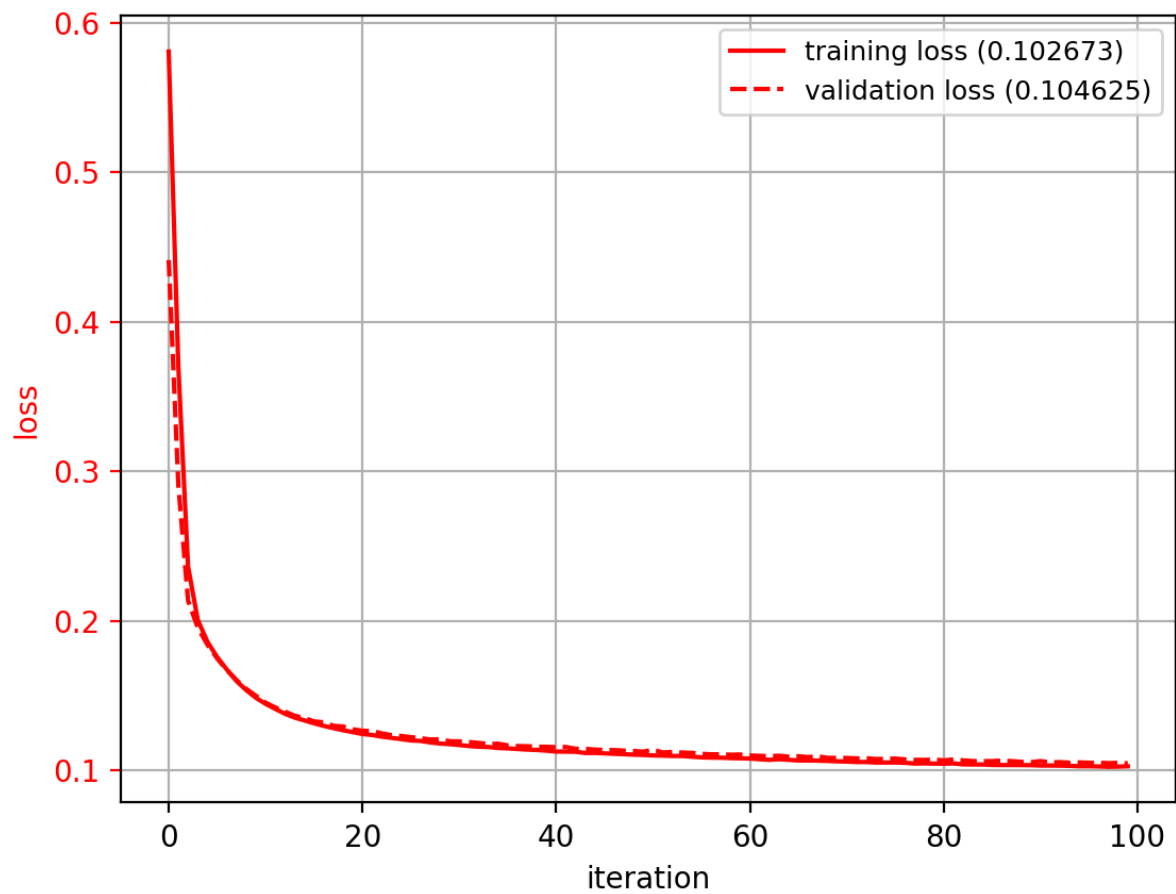
Epoch 1/100: Train Loss: 0.580738 Val Loss: 0.441278
Epoch 2/100: Train Loss: 0.369752 Val Loss: 0.290536
Epoch 3/100: Train Loss: 0.236372 Val Loss: 0.213465
Epoch 4/100: Train Loss: 0.200582 Val Loss: 0.195443
Epoch 5/100: Train Loss: 0.185938 Val Loss: 0.183953
Epoch 6/100: Train Loss: 0.175762 Val Loss: 0.174883
Epoch 7/100: Train Loss: 0.167399 Val Loss: 0.167010
Epoch 8/100: Train Loss: 0.159727 Val Loss: 0.160395
Epoch 9/100: Train Loss: 0.153644 Val Loss: 0.154492
Epoch 10/100: Train Loss: 0.148495 Val Loss: 0.149854
Epoch 11/100: Train Loss: 0.144461 Val Loss: 0.145233
Epoch 12/100: Train Loss: 0.141093 Val Loss: 0.142052
Epoch 13/100: Train Loss: 0.137806 Val Loss: 0.139236
Epoch 14/100: Train Loss: 0.135250 Val Loss: 0.136462
Epoch 15/100: Train Loss: 0.133388 Val Loss: 0.134951
Epoch 16/100: Train Loss: 0.131438 Val Loss: 0.132702
Epoch 17/100: Train Loss: 0.129716 Val Loss: 0.132089
Epoch 18/100: Train Loss: 0.128198 Val Loss: 0.129765
Epoch 19/100: Train Loss: 0.126775 Val Loss: 0.129258
Epoch 20/100: Train Loss: 0.125562 Val Loss: 0.127433
Epoch 21/100: Train Loss: 0.124283 Val Loss: 0.126425
Epoch 22/100: Train Loss: 0.123612 Val Loss: 0.126277
Epoch 23/100: Train Loss: 0.122492 Val Loss: 0.124672
Epoch 24/100: Train Loss: 0.121693 Val Loss: 0.123506
Epoch 25/100: Train Loss: 0.120852 Val Loss: 0.122943
Epoch 26/100: Train Loss: 0.119977 Val Loss: 0.122155
Epoch 27/100: Train Loss: 0.119669 Val Loss: 0.121492
Epoch 28/100: Train Loss: 0.118649 Val Loss: 0.120575
Epoch 29/100: Train Loss: 0.117905 Val Loss: 0.120805
EarlyStopping counter: 1 out of 5
Epoch 30/100: Train Loss: 0.117491 Val Loss: 0.119440
Epoch 31/100: Train Loss: 0.116906 Val Loss: 0.119315
Epoch 32/100: Train Loss: 0.116209 Val Loss: 0.118878
Epoch 33/100: Train Loss: 0.115755 Val Loss: 0.118320
Epoch 34/100: Train Loss: 0.115578 Val Loss: 0.117556
Epoch 35/100: Train Loss: 0.114895 Val Loss: 0.117774
EarlyStopping counter: 1 out of 5
Epoch 36/100: Train Loss: 0.114666 Val Loss: 0.116668
Epoch 37/100: Train Loss: 0.114253 Val Loss: 0.116196
Epoch 38/100: Train Loss: 0.113844 Val Loss: 0.116079
Epoch 39/100: Train Loss: 0.113595 Val Loss: 0.115725
Epoch 40/100: Train Loss: 0.113128 Val Loss: 0.115701
EarlyStopping counter: 1 out of 5

```

Epoch 41/100:	Train Loss: 0.112555	Val Loss: 0.115428
Epoch 42/100:	Train Loss: 0.112481	Val Loss: 0.115522
EarlyStopping counter: 1 out of 5		
Epoch 43/100:	Train Loss: 0.112405	Val Loss: 0.114390
Epoch 44/100:	Train Loss: 0.111570	Val Loss: 0.114342
EarlyStopping counter: 1 out of 5		
Epoch 45/100:	Train Loss: 0.111643	Val Loss: 0.113861
Epoch 46/100:	Train Loss: 0.111325	Val Loss: 0.113519
Epoch 47/100:	Train Loss: 0.110998	Val Loss: 0.113524
EarlyStopping counter: 1 out of 5		
Epoch 48/100:	Train Loss: 0.110776	Val Loss: 0.113125
Epoch 49/100:	Train Loss: 0.110449	Val Loss: 0.112895
Epoch 50/100:	Train Loss: 0.110235	Val Loss: 0.112461
Epoch 51/100:	Train Loss: 0.109955	Val Loss: 0.113816
EarlyStopping counter: 1 out of 5		
Epoch 52/100:	Train Loss: 0.109821	Val Loss: 0.111919
Epoch 53/100:	Train Loss: 0.109585	Val Loss: 0.112235
EarlyStopping counter: 1 out of 5		
Epoch 54/100:	Train Loss: 0.109549	Val Loss: 0.111859
EarlyStopping counter: 2 out of 5		
Epoch 55/100:	Train Loss: 0.109088	Val Loss: 0.111631
Epoch 56/100:	Train Loss: 0.108672	Val Loss: 0.111025
Epoch 57/100:	Train Loss: 0.108485	Val Loss: 0.110649
Epoch 58/100:	Train Loss: 0.108442	Val Loss: 0.110739
EarlyStopping counter: 1 out of 5		
Epoch 59/100:	Train Loss: 0.108136	Val Loss: 0.110408
Epoch 60/100:	Train Loss: 0.108022	Val Loss: 0.110542
EarlyStopping counter: 1 out of 5		
Epoch 61/100:	Train Loss: 0.107865	Val Loss: 0.109884
Epoch 62/100:	Train Loss: 0.107378	Val Loss: 0.109669
Epoch 63/100:	Train Loss: 0.106982	Val Loss: 0.109430
Epoch 64/100:	Train Loss: 0.107399	Val Loss: 0.109696
EarlyStopping counter: 1 out of 5		
Epoch 65/100:	Train Loss: 0.107010	Val Loss: 0.109132
Epoch 66/100:	Train Loss: 0.106557	Val Loss: 0.109390
EarlyStopping counter: 1 out of 5		
Epoch 67/100:	Train Loss: 0.106543	Val Loss: 0.108853
Epoch 68/100:	Train Loss: 0.106486	Val Loss: 0.109368
EarlyStopping counter: 1 out of 5		
Epoch 69/100:	Train Loss: 0.106265	Val Loss: 0.108320
Epoch 70/100:	Train Loss: 0.106022	Val Loss: 0.108271
EarlyStopping counter: 1 out of 5		
Epoch 71/100:	Train Loss: 0.105827	Val Loss: 0.108218
Epoch 72/100:	Train Loss: 0.105525	Val Loss: 0.107946
Epoch 73/100:	Train Loss: 0.105545	Val Loss: 0.108071
EarlyStopping counter: 1 out of 5		
Epoch 74/100:	Train Loss: 0.105175	Val Loss: 0.107541
Epoch 75/100:	Train Loss: 0.105197	Val Loss: 0.107617
EarlyStopping counter: 1 out of 5		
Epoch 76/100:	Train Loss: 0.105213	Val Loss: 0.107839
EarlyStopping counter: 2 out of 5		
Epoch 77/100:	Train Loss: 0.105007	Val Loss: 0.107246

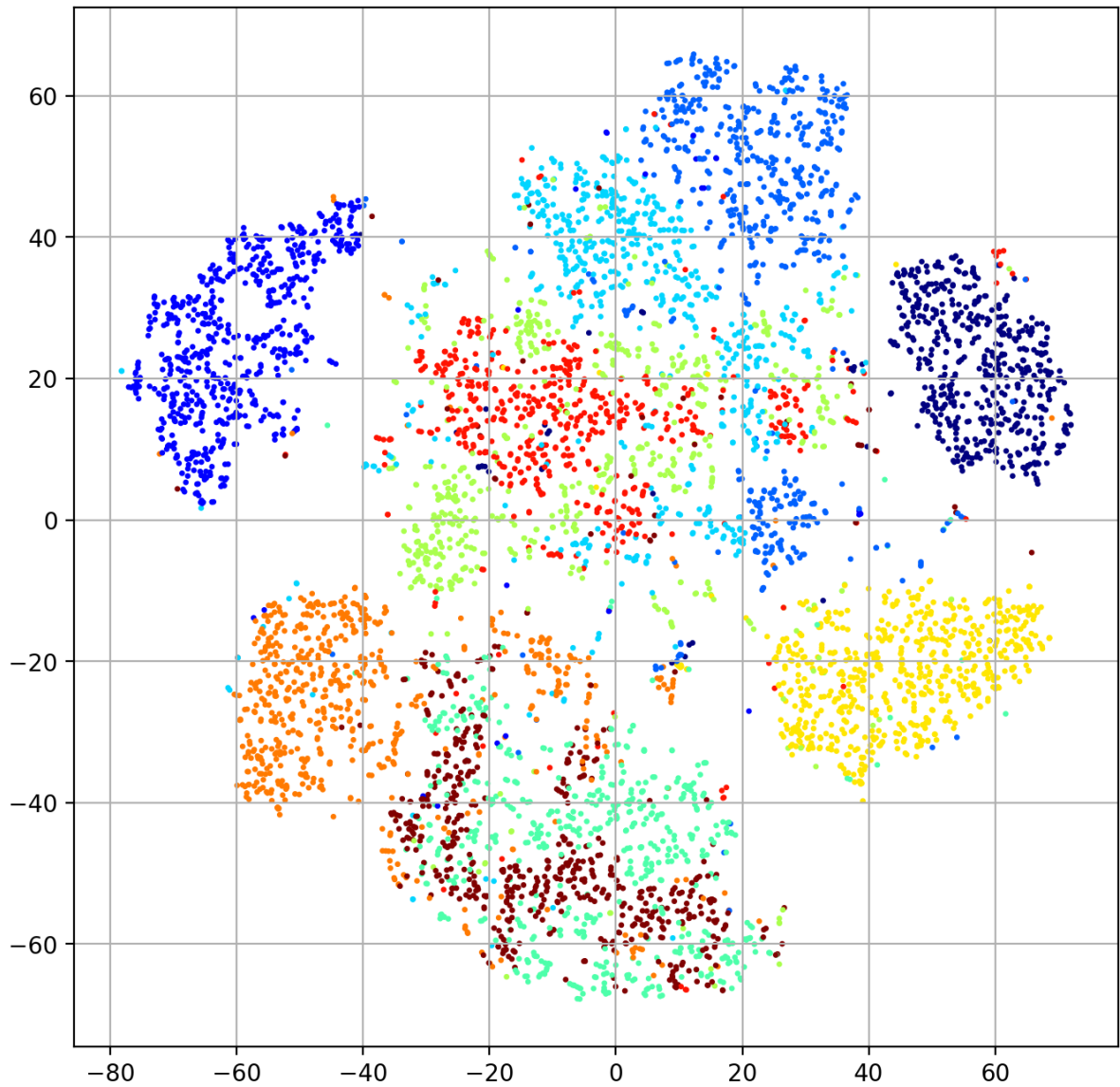
```
Epoch 78/100:   Train Loss: 0.104496   Val Loss: 0.106999
Epoch 79/100:   Train Loss: 0.104632   Val Loss: 0.106901
EarlyStopping counter: 1 out of 5
Epoch 80/100:   Train Loss: 0.104474   Val Loss: 0.107186
EarlyStopping counter: 2 out of 5
Epoch 81/100:   Train Loss: 0.104388   Val Loss: 0.106633
Epoch 82/100:   Train Loss: 0.104591   Val Loss: 0.107214
EarlyStopping counter: 1 out of 5
Epoch 83/100:   Train Loss: 0.103965   Val Loss: 0.106620
EarlyStopping counter: 2 out of 5
Epoch 84/100:   Train Loss: 0.103871   Val Loss: 0.106178
Epoch 85/100:   Train Loss: 0.103893   Val Loss: 0.106119
EarlyStopping counter: 1 out of 5
Epoch 86/100:   Train Loss: 0.103579   Val Loss: 0.106149
EarlyStopping counter: 2 out of 5
Epoch 87/100:   Train Loss: 0.103504   Val Loss: 0.106469
EarlyStopping counter: 3 out of 5
Epoch 88/100:   Train Loss: 0.103524   Val Loss: 0.106135
EarlyStopping counter: 4 out of 5
Epoch 89/100:   Train Loss: 0.103481   Val Loss: 0.105626
Epoch 90/100:   Train Loss: 0.103418   Val Loss: 0.105435
Epoch 91/100:   Train Loss: 0.103177   Val Loss: 0.106125
EarlyStopping counter: 1 out of 5
Epoch 92/100:   Train Loss: 0.103181   Val Loss: 0.105288
Epoch 93/100:   Train Loss: 0.103117   Val Loss: 0.105441
EarlyStopping counter: 1 out of 5
Epoch 94/100:   Train Loss: 0.102972   Val Loss: 0.105043
Epoch 95/100:   Train Loss: 0.102694   Val Loss: 0.105252
EarlyStopping counter: 1 out of 5
Epoch 96/100:   Train Loss: 0.102707   Val Loss: 0.105001
EarlyStopping counter: 2 out of 5
Epoch 97/100:   Train Loss: 0.102564   Val Loss: 0.105012
EarlyStopping counter: 3 out of 5
Epoch 98/100:   Train Loss: 0.102320   Val Loss: 0.104782
Epoch 99/100:   Train Loss: 0.102441   Val Loss: 0.104942
EarlyStopping counter: 1 out of 5
Epoch 100/100:  Train Loss: 0.102673   Val Loss: 0.104625
```

```
In [40]: plot_history(history)
```

- Visualize the latent space

```
In [41]: model.eval()  
Z = model.encoder(trainI).flatten(1).cpu().numpy()  
plot_embedding(Z, trainY, perplexity=30.)
```



- Encode and reconstruct an image

```
In [42]: testIrecon = model(testI.float().to(device))
```

- Better visualization

```
In [43]: imglist = []
for j,i in enumerate(range(0,100,10)):
    tmp = np.hstack( (testI[i].reshape((28,28)).numpy()*255.0, testIrecon[i].reshape((28,28)).numpy()*255.0)
    imglist.append(tmp)
rfig = plt.figure(figsize=(10,3))
show_imgs(imglist,nc=5)
plt.close()
```



- Traverse the latent space
 - Sample points:

$$\mathbf{z}_1 = f(\mathbf{x}_1), \mathbf{z}_2 = f(\mathbf{x}_2)$$

- interpolate:

$$\hat{\mathbf{z}} = (1 - \alpha)\mathbf{z}_1 + \alpha\mathbf{z}_2, 0 \leq \alpha \leq 1$$

- reconstruct:

$$\hat{\mathbf{x}} = g(\hat{\mathbf{z}})$$

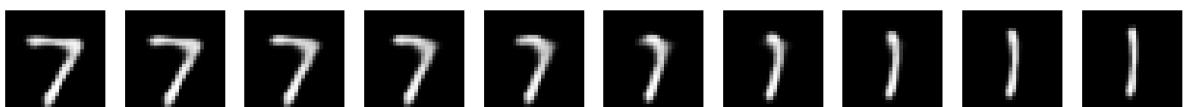
- Change a 7 into a 1

```
In [44]: def Xinterp(X, sp=10):
          alpha = np.linspace(0, 1, sp).reshape((sp,) + (np.ndim(X) - 1) * (1,))
          Xint = X[0, :] * (1 - alpha) + X[1, :] * alpha
          return Xint
```

```
In [45]: inds = [0, 40]
          X = model.encoder(testI[inds, :].float().to(device))
          Xd = model.decoder(Xinterp(X).float().to(device))
```

```
/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_75225/119390
0626.py:3: DeprecationWarning: __array_wrap__ must accept context and r
eturn_scalar arguments (positionally) in the future. (Deprecated NumPy
2.0)
          Xint = X[0, :] * (1 - alpha) + X[1, :] * alpha
```

```
In [46]: rfig = plt.figure(figsize=(10, 3))
          show_imgs(Xd.reshape((10, 28, 28)).numpy() * 255.0, nc=10)
          plt.close()
```



- Traverse the latent space between a 7, 1, 9, and 4

- captures shapes in between

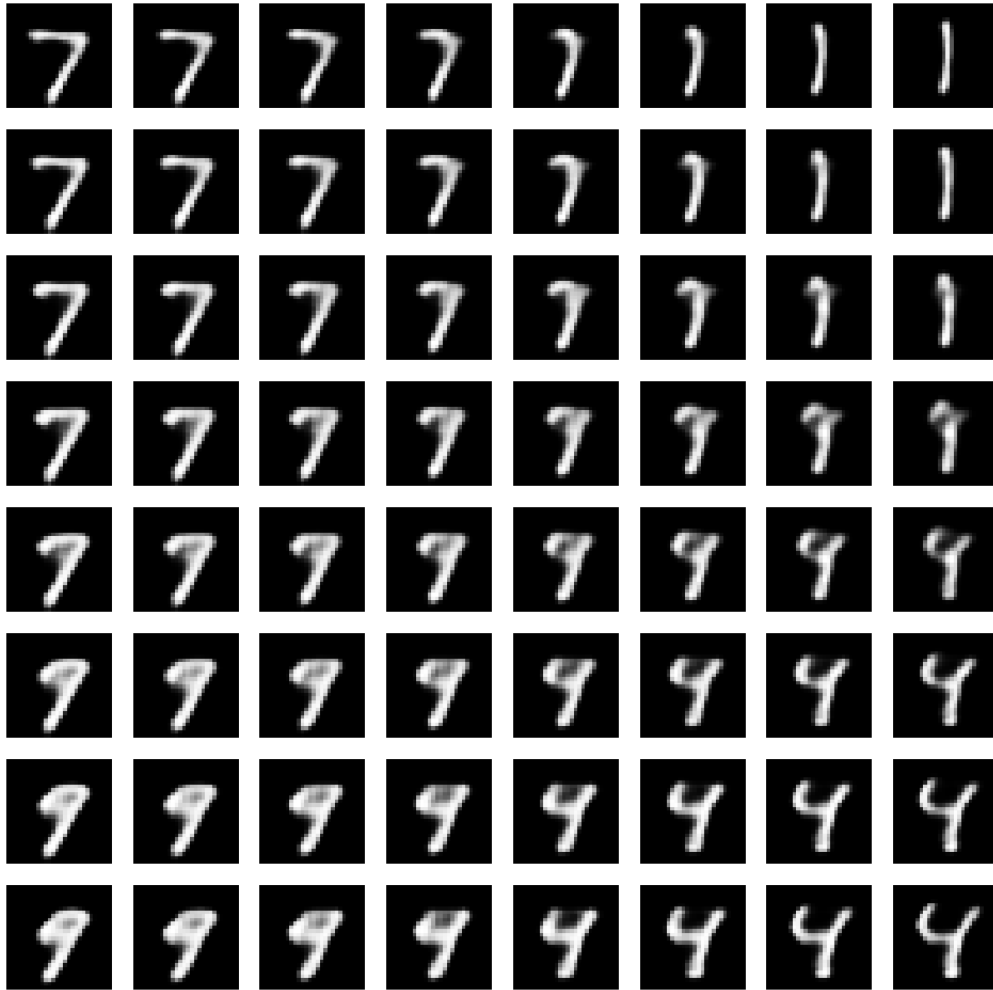
```
In [47]: def Xinterp2(X, sp=10):
          Xtbl = Xinterp(X[[0,2],:], sp=sp)
          Xtbr = Xinterp(X[[1,3],:], sp=sp)

          # interpolate inbetween
          Xall = torch.zeros((sp*sp,) + Xtbl.shape[1:])
          for i in range(sp):
              tmpX = torch.concat((Xtbl[i,:].unsqueeze(0), Xtbr[i,:].unsqueeze(0)), 0)
              Xall[sp*i:sp*(i+1),:] = Xinterp(tmpX, sp=sp)
          return Xall
```

```
In [48]: inds = [0, 40, 20, 6]
          X = model.encoder(testI[inds,:].float().to(device))
          Xd = model.decoder(Xinterp2(X, sp=8).float().to(device))
          rfig = plt.figure(figsize=(6,6))
          show_imgs(Xd.reshape((64,28,28)).numpy()*255.0,nc=8)
          plt.close()
```

/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_75225/1193900626.py:3: DeprecationWarning: __array_wrap__ must accept context and return_scalar arguments (positionally) in the future. (Deprecated NumPy 2.0)

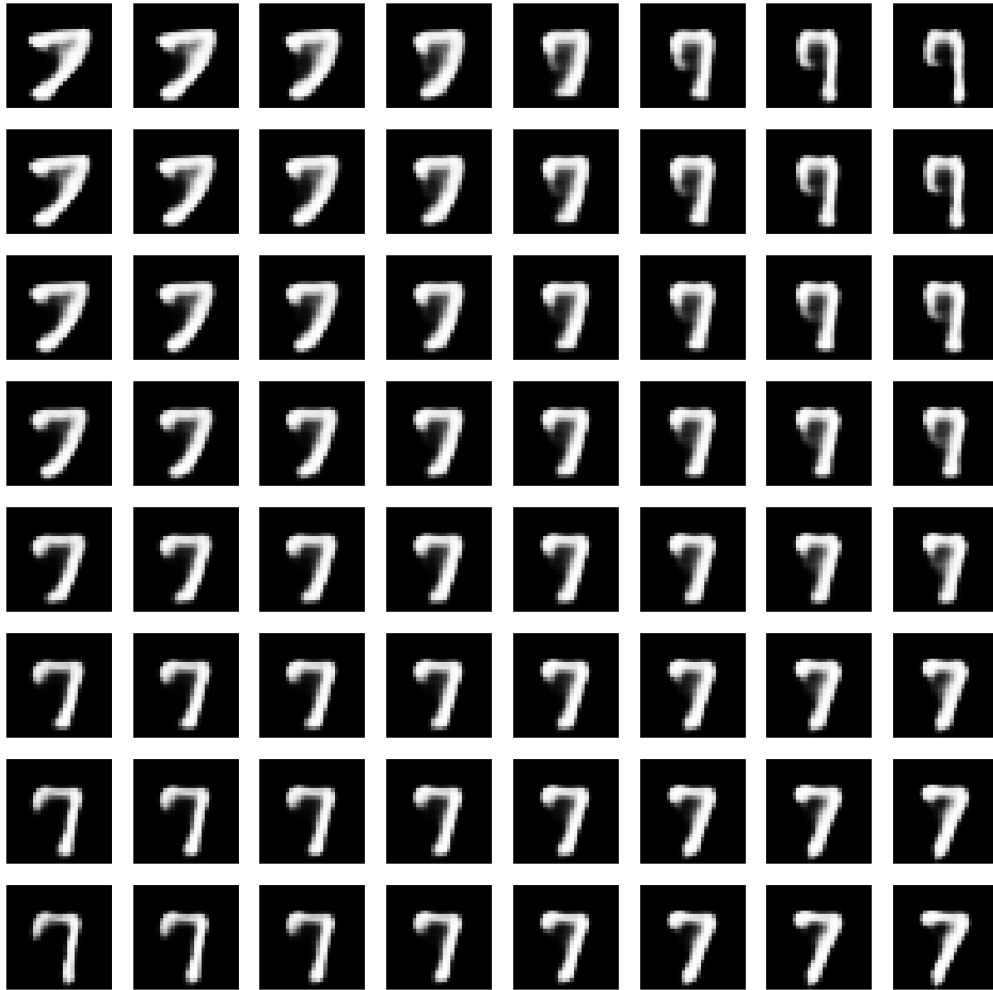
```
Xint = X[0,:]*(1-alpha) + X[1,:]*alpha
```



- Traverse the latent space between different 7s
 - captures different shapes of 7

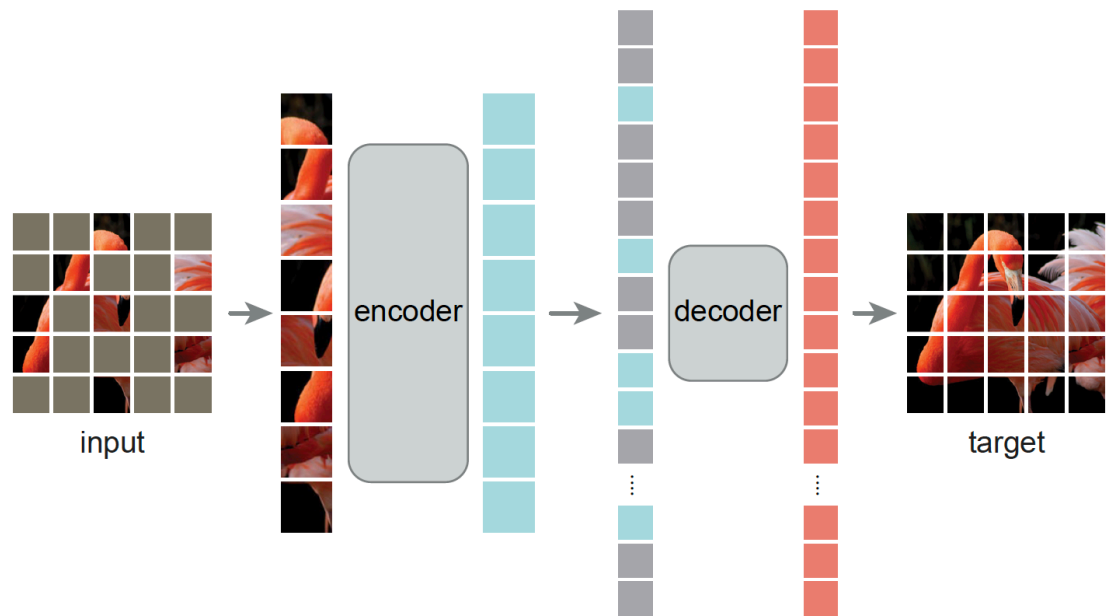
```
In [49]: mydigits = np.where(testY==7)[0]
         inds = mydigits[10:14]
         X = model.encoder(testI[inds,:].float().to(device))
         Xd = model.decoder(Xinterp2(X,sp=8).float().to(device))
         rfig = plt.figure(figsize=(6,6))
         show_imgs(Xd.reshape((64,28,28)).numpy()*255.0,nc=8)
         plt.close()
```

```
/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_75225/119390
0626.py:3: DeprecationWarning: __array_wrap__ must accept context and r
eturn_scalar arguments (positionally) in the future. (Deprecated NumPy
2.0)
     Xint = X[0,:]*(1-alpha) + X[1,:]*alpha
```



Masked Autoencoder (MAE)

- Combine ViT with auto-encoder for self-supervised representation learning.
 - encode subset of input patches (mask out 75%)
 - decode encoded & masked patches to reconstruct the image
- Pre-train on IN1k (w/o labels)



- Fine-tune on IN1k classification task
 - better than supervised pre-training of ViT with IN1k
 - but worse than supervised pre-training with large-scale data (JFT)

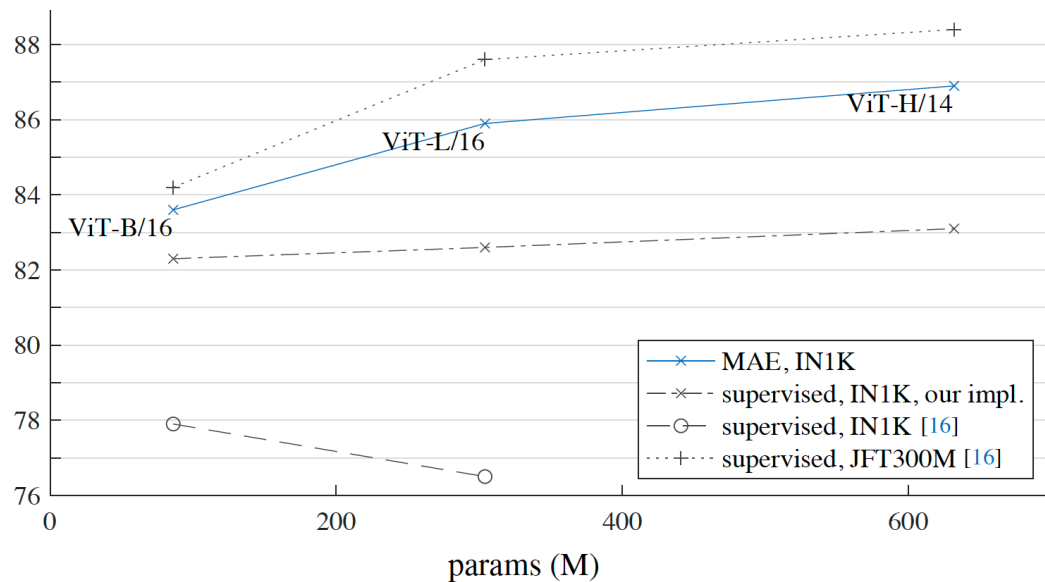


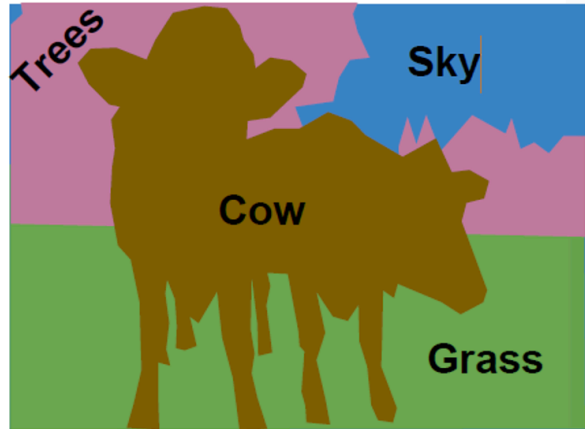
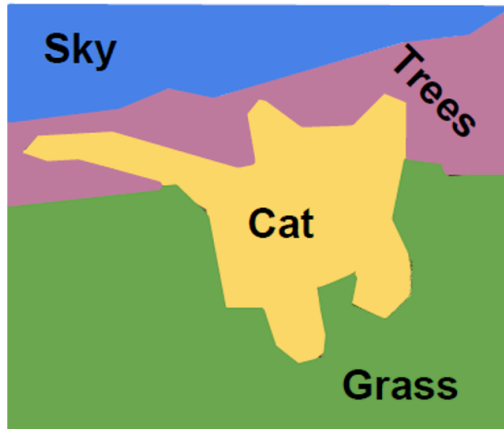
Figure 8. **MAE pre-training vs. supervised pre-training**, evaluated by fine-tuning in ImageNet-1K (224 size). We compare with the original ViT results [16] trained in IN1K or JFT300M.

Semantic Segmentation

- Label each pixel in the image with a category label.

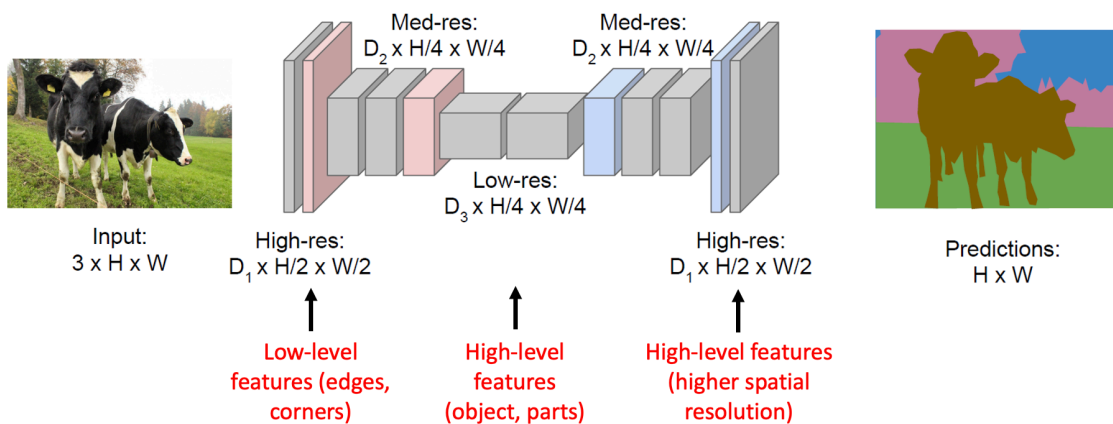


This image is CC0 public domain



Fully-convolutional network (FCN)

- learns mapping from input image to output segmentation
 - encoder/decoder structure similar to AE.
 - middle contains the low-dim representation.



U-net: More sophisticated FCN

- Idea: concatenate higher-resolution features when upsampling

- Better localize boundaries.
- Mix high-level and low-level features.

